

UNIT-1

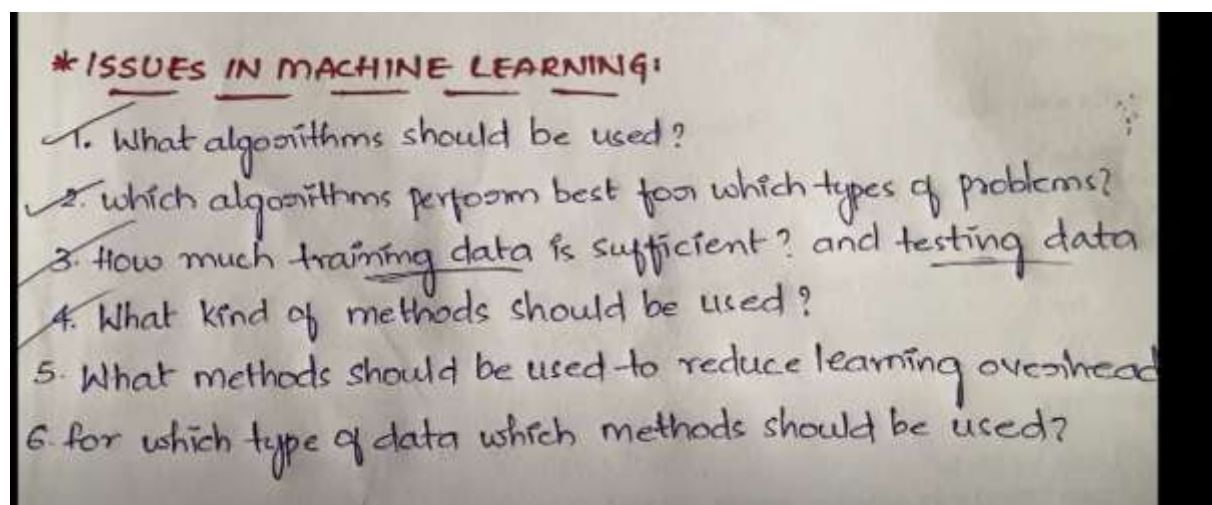
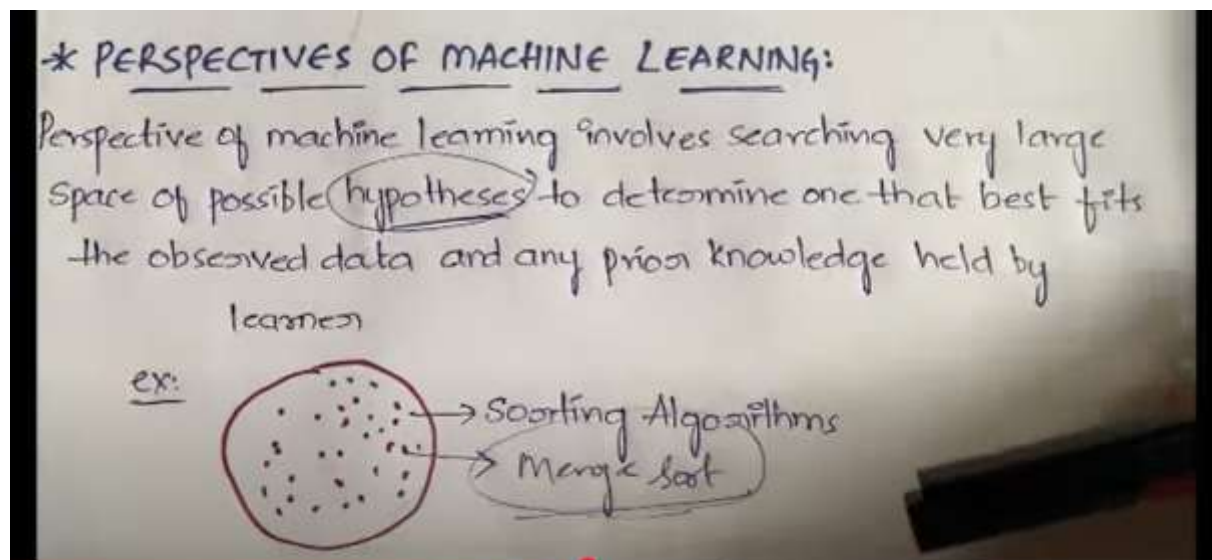
1.Types of Machine Learning

<https://www.geeksforgeeks.org/types-of-machine-learning/>

2.Design a Machine Learning System(brain and neuron design of Learning system)

<https://huyenchip.com/machine-learning-systems-design/design-a-machine-learning-system.html>

3.Perspective and Issue in Machine Learning



4. Concept Learning Task

- **A CONCEPT LEARNING TASK:**
- Each hypothesis be a vector of six constraints, specifying the values of the six attributes *Sky*, *AirTemp*, *Humidity*, *Wind*, *Water*, and *Forecast*. For each attribute, the hypothesis will either indicate by a "?" that any value is acceptable for this attribute, specify a single required value (e.g., *Warm*) for the attribute, or indicate by a "0" that no value is acceptable.

Sky	Temp	Humid	Wind	Water	Forecst	EnjoySpt
Sunny	Warm	Normal	Strong	Warm	Same	Yes
Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	High	Strong	Cool	Change	Yes

- The most general hypothesis-that every day is a positive example-is represented by
 - (?, ?, ?, ?, ?, ?)
 - and the most specific possible hypothesis-that no day is a positive example-is represented by negative example
 - (0,0,0,0,0,0)
-
- If some instance x satisfies all the constraints of hypothesis h , then h classifies x as a positive example ($h(x) = 1$).
 - To illustrate, the hypothesis that Aldo enjoys his favourite sport only on cold days with high humidity (independent of the values of the other attributes) is represented by the expression
 - (?, *Cold*, High, ?, ?, ?)

5. concept learning as search in machine learning

* CONCEPT LEARNING AS SEARCH:

Main goal of this search is to find the hypothesis that best fits the training examples.

Examples:

EnjoySport learning task

6 attributes
 (Sky, Air temperature, humidity, wind, water, and forecast)

3 values - rainy, cloudy and sunny

Remaining attributes - only 2 values

3 values - rainy, cloudy and sunny

Remaining attributes - only 2 values

∴ Different instances possible = $(3 \times 2 \times 2 \times 2 \times 2 \times 2) = 96$

✓ Syntactically distinct hypothesis = $5 \times 4 \times 4 \times 4 \times 4 \times 4 = 5120$

(Additionally, 2 more values - ? and ϕ)

Semantically distinct hypothesis = $1 + (4 \times 3 \times 3 \times 3 \times 3 \times 3)$

(Null - taken as common)

(ϕ) - only 1 more value)

≈ 973

Semantically distinct hypothesis = $1 + (4 \times 3 \times 3 \times 3 \times 3 \times 3)$

(Null - taken as common)

(ϕ) - only 1 more value)

≈ 973

- After finding all the syntactically & semantically distinct hypothesis,

we search the best match from all these

(i.e. much closer to our learning problem)

6. Version Space

<https://medium.com/%40yashi11/version-space-learning-for-ml-259d8423e146>

* VERSION SPACE:

Subset of hypothesis H consistent with the training examples

$$VS_{H,D} = \{h \in H \mid \text{consistent}(h,D)\}$$

H = hypothesis

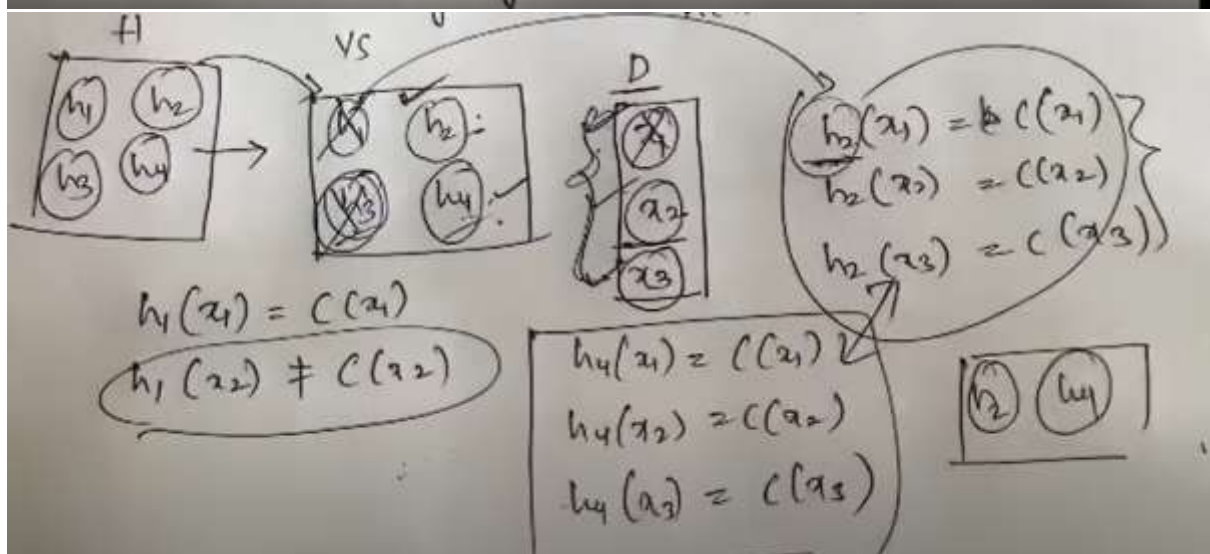
D = training examples

consistent
 $h(x) = c(x)$

* Algorithm to obtain Version Space:

(List then eliminate algorithm) $\rightarrow$ ③

1. Version Space $\leftarrow$ list containing every hypothesis in H
2. from this step, we keep on removing inconsistent hypothesis from version space
for each training example, $\langle x, c(x) \rangle$ remove any hypothesis that is $h(x) \neq c(x)$
3. dp the list of hypothesis into version space after checking for all training examples.

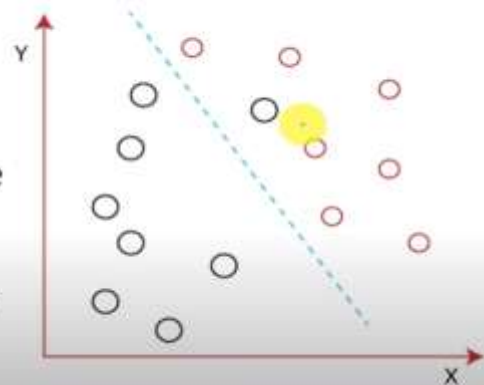


Linear Discriminant Analysis - Introduction

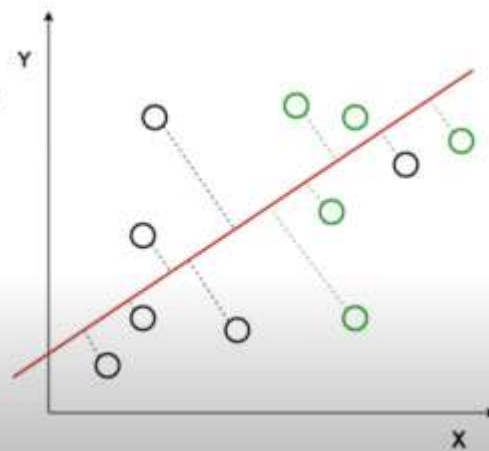
- Linear Discriminant Analysis or Normal Discriminant Analysis or Discriminant Function Analysis is a dimensionality reduction technique that is commonly used for supervised classification problems.
- It is used to project the features in higher dimension space into a lower dimension space.

- Suppose we have two sets of data points belonging to two different classes that we want to classify.

- When the data points are plotted on the 2D plane, there's no straight line that can separate the two classes of the data points completely.



- Here, Linear Discriminant Analysis uses both the axes (X and Y) to create a new axis and projects data onto a new axis in a way to maximize the separation of the two categories and hence, reducing the 2D graph into a 1D graph.



- Two criteria are used by LDA to create a new axis:
 - Maximize the distance between means of the two classes.
 - Minimize the variation within each class.



1. Compute the class means of dependent variable
2. Derive the covariance matrix of the class variable
3. Compute the within class — scatter matrix ($S_1 + S_2$)
4. Compute the between class scatter matrix
5. Compute the Eigen values and eigen vectors from the within class and between class scatter matrix

$$\mu_i = \frac{1}{N_i} \sum_{x \in \omega_i} x$$

$$S_i = \sum_{x \in \omega_i} (x - \mu_i)(x - \mu_i)^T$$

$$S_w = S_1 + S_2$$

$$S_B = (\mu_1 - \mu_2)(\mu_1 - \mu_2)^T$$

$$S_w^{-1} S_B w = \lambda w$$

6. Sort the values of eigen values and select the top k values
7. Find the eigen vectors corresponds to the top k eigen vectors
8. Obtain the LDA by taking the dot product of eigen vectors and original data

$$\left(S_w^{-1} S_B - \lambda I \right) \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} = 0$$

Perceptron Learning – Introduction

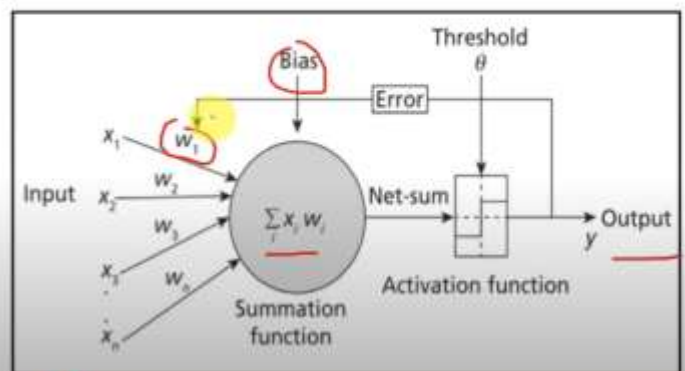
- The first neural network model 'Perceptron', designed in 1958
- Perceptron is a linear binary classifier used for supervised learning.
- Perceptron Learning model is a combination of two concepts,

McCulloch-Pitts model of an artificial neuron and Hebbian learning rule of adjusting weights.

The perceptron model consists of 4 steps:

1. Inputs from other neurons
2. Weights and bias
3. Net sum
4. Activation function

$$Y = \begin{cases} 1 & \text{if } f(x) \geq \theta \\ 0 & \text{if } f(x) < \theta \end{cases}$$



Set initial weights $w_1, w_2, \dots, w_n$ and bias θ to a random value in the range $[-0.5, 0.5]$.

For each Epoch,

1. Compute the weighted sum by multiplying the inputs with the weights and add the products.
2. Apply the activation function on the weighted sum:

$$Y = \text{Step}((x_1 w_1 + x_2 w_2) - \theta)$$
3. If the sum is above the threshold value, output the value as positive else output the value as negative.
4. Calculate the error by subtracting the estimated output $Y_{\text{estimated}}$ from the desired output Y_{desired} :

$$\text{error } e(t) = Y_{\text{desired}} - Y_{\text{estimated}}$$

5. Update the weights if there is an error:

$$\Delta w_i = \alpha \times e(t) \times x_i$$

$$w_i = w_i + \Delta w_i$$

where, x_i is the input value, $e(t)$ is the error at step t , α is the learning rate and Δw_i is the difference in weight that has to be added to w_i .

9. Linear Regression

Linear regression : (5)

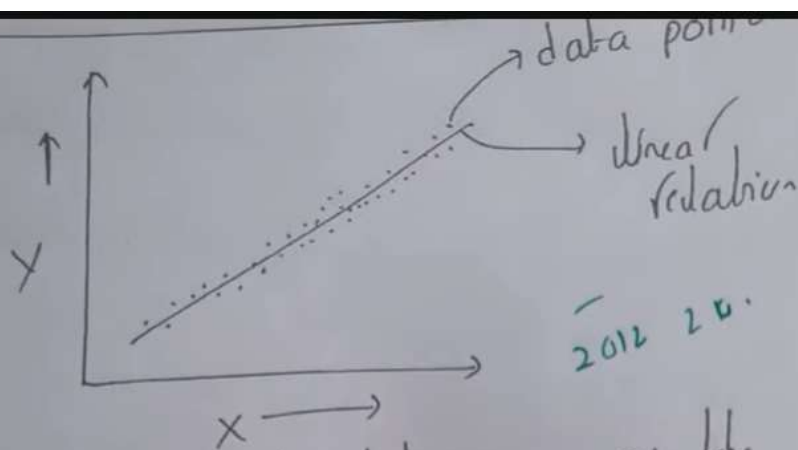
→ Linear regression is a machine learning algorithm based on supervised learning.

→ It performs regression task.

→ In linear regression perform the task to predict dependent variable value (y) based on a given independent variable (x). So this

perform the task to predict dependent variable value (y) based on a given independent variable (x). So this regression technique find out a linear relation between x (input) & y (output). Hence the name is linear regression.

→ dependent variable is continuous in nature.



(linear indirectly means as the value of x is increasing value of y is also increasing)

→ So based on the no. of independent variables linear regression is divided into 2 types

y is also increasing
→ So based on the no. of independent variables linear regression is divided into 2 types

i) Simple linear eqⁿ

$$y = \alpha_0 + \alpha_1 x_1$$

or

$$y = c + m x$$

ii) multiple linear eqⁿ

$$y = \alpha_0 + \alpha_1 x_1 + \alpha_2 x_2 + \dots + \alpha_n x_n$$

$m \rightarrow$ slope
 $c \rightarrow$ intercept

$y \rightarrow$ dependent

$x \rightarrow$ independent

$\alpha_i \rightarrow$ regression coeff.

→ Regression coefficient are collected at training based on the data set based on the impact of an independent variable value on the final result/output.
→ Hence if a variable's co-efficient-

→ Hence if a variable's co-efficient is high we can directly say that changing its value will impact a higher change in the final output.

ex $y = 0.9 + 1.2x_1 + 2x_2 + 4x_3 + 1x_4$

here among all 4 is the highest co-efficient so based on value x_3 the output depends a lot.

10. Artificial Neural Network

<https://www.geeksforgeeks.org/artificial-neural-networks-and-its-applications/>

11. McCulloch-pits Neuron and Hebb Network

<https://www.linkedin.com/pulse/mcculloch-pitts-neuron-hebb-network-himanshu-salunke-amaof/>

12. Linear Separability

<https://www.youtube.com/watch?v=vwrOclhy5k0>

13. Find S-Algorithm (Find a maximally specific hypothesis)

FIND-S: Step-1

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

1. Initialize h to the most specific hypothesis in H vtupulse.com

$h_0 = \langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle$

Mahesh Huddar

FIND-S: Step-2

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

2. For each positive training instance x

- For each attribute constraint a_i in h

If the constraint a_i is satisfied by x

Then do nothing

Else replace a_i in h by the next more general constraint that is satisfied by x

$h_0 = \langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle$

$x_1 = \langle \text{Sunny, Warm, Normal, Strong, Warm, Same} \rangle$

Iteration 1

$h_1 = \langle \text{Sunny, Warm, Normal, Strong, Warm, Same} \rangle$

Mahesh Huddar

vtupulse.com

FIND-S: Step-2

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

Iteration 2

h1 = <Sunny, Warm, Normal, Strong, Warm, Same>

x2 = <Sunny, Warm, High, Strong, Warm, Same>

h2 = <Sunny, Warm, ?, Strong, Warm, Same>

vtupulse.com

Mahesh Huddar

FIND-S: Step-2

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

Iteration 3

Ignore

h3 = <Sunny, Warm, ?, Strong, Warm, Same>

vtupulse.com

Mahesh Huddar

FIND-S: Step-2

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

Iteration 4

h3 = <Sunny, Warm, ?, Strong, Warm, Same>

x4 = <Sunny, Warm, High, Strong, Cool, Change>

Step 3

Output

h4 = <Sunny, Warm, ?, Strong, ?, ?>

vtupulse.com

Mahesh Huddar

14.Candidate Elimination Algo

<https://www.youtube.com/watch?v=O2wYwFOMQ24>

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

$S_0:$ $\langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle$
 $S_1:$ $\langle \text{Sunny}, \text{Warm}, \text{Normal}, \text{Strong}, \text{Warm}, \text{Same} \rangle$
 $S_2:$ $S_3:$ $\langle \text{Sunny}, \text{Warm}, ?, \text{Strong}, \text{Warm}, \text{Same} \rangle$

vtupulse.com

$G_3:$ $\langle \text{Sunny}, ?, ?, ?, ?, ? \rangle$ $\langle ?, \text{Warm}, ?, ?, ?, ? \rangle$ $\langle ?, ?, \text{Normal}, ?, ?, ? \rangle$ $\langle ?, ?, ?, ?, \text{Cool}, ? \rangle$ $\langle ?, ?, ?, ?, ?, \text{Same} \rangle$
 $G_0:$ $G_1:$ $G_2:$ $\langle ?, ?, ?, ?, ?, ? \rangle$

Mahesh Huddar

Learned Version Space by Candidate Elimination Algorithm

S $\langle \text{Sunny}, \text{Warm}, ?, \text{Strong}, ?, ? \rangle$

$\langle \text{Sunny}, ?, ?, \text{Strong}, ?, ? \rangle$ $\langle \text{Sunny}, \text{Warm}, ?, ?, ?, ? \rangle$ $\langle ?, \text{Warm}, ?, \text{Strong}, ?, ? \rangle$

vtupulse.com

G $\langle \text{Sunny}, ?, ?, ?, ?, ? \rangle$ $\langle ?, \text{Warm}, ?, ?, ?, ? \rangle$

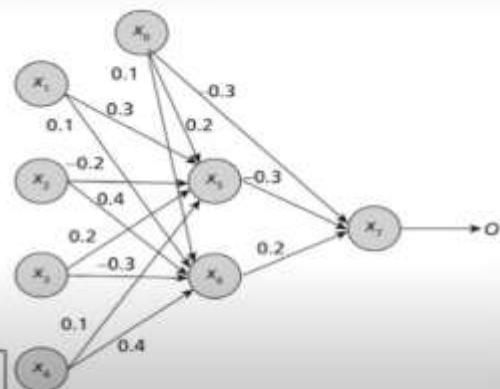
Mahesh Huddar

Unit-2

1.MLP

- The given MLP consists of an Input layer, one Hidden layer and an Output layer.
- The input layer has 4 neurons, the hidden layer has 2 neurons and the output layer has a single neuron.
- Train the MLP by updating the weights and biases in the network.
- Learning Rate = 0.8

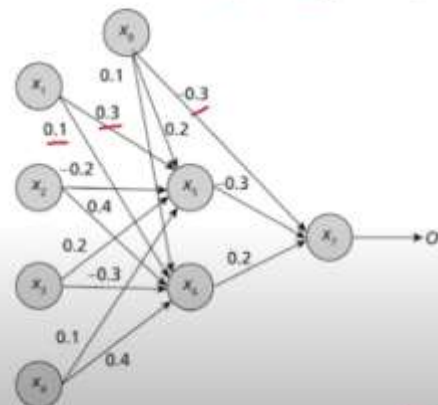
x_1	x_2	x_3	x_4	$O_{Desired}$
1	1	0	1	1



x_1	x_2	x_3	x_4	w_{15}	w_{16}	w_{25}	w_{26}	w_{35}	w_{36}	w_{45}	w_{46}	w_{57}	w_{67}	θ_5	θ_6	θ_7
1	1	0	1	0.3	0.1	-0.2	0.4	0.2	-0.3	0.1	0.4	-0.3	0.2	0.2	0.1	-0.3

- Calculate Input and Output in the Input Layer
Net Input and Output Calculation

Input layer	I_i	O_i
x_1	1	1
x_2	1	1
x_3	0	0
x_4	1	1



x_1	x_2	x_3	x_4	w_{15}	w_{16}	w_{25}	w_{26}	w_{35}	w_{36}	w_{45}	w_{46}	w_{57}	w_{67}	θ_5	θ_6	θ_7
1	1	0	1	0.3	0.1	-0.2	0.4	0.2	-0.3	0.1	0.4	-0.3	0.2	0.2	0.1	-0.3

2. Calculate Net Input and Output in the Hidden Layer and Output Layer as

Unit _j	Net Input I_j	Output O_j
x_5	$I_5 = x_1 \times w_{15} + x_2 \times w_{25} + x_3 \times w_{35} + x_4 \times w_{45} + x_0 \times \theta_5$ $I_5 = 1 \times 0.3 + 1 \times -0.2 + 0 \times 0.2 + 1 \times 0.1 + 1 \times 0.2 = 0.4$	$O_5 = \frac{1}{1 + e^{-I_5}} = \frac{1}{1 + e^{-0.4}} = 0.599$
x_6	$I_6 = x_1 \times w_{16} + x_2 \times w_{26} + x_3 \times w_{36} + x_4 \times w_{46} + x_0 \times \theta_6$ $I_6 = 1 \times 0.3 + 1 \times 0.4 + 0 \times -0.3 + 1 \times 0.4 + 1 \times 0.1 = 1.2$	$O_6 = \frac{1}{1 + e^{-I_6}} = \frac{1}{1 + e^{-1.2}} = 0.769$
x_7	$I_7 = O_5 \times w_{57} + O_6 \times w_{67} + x_0 \times \theta_7$ $I_7 = 0.599 \times -0.3 + 0.769 \times 0.2 + 1 \times -0.3 = -0.326$	$O_7 = \frac{1}{1 + e^{-I_7}} = \frac{1}{1 + e^{-0.326}} = 0.419$

Multi-Layer Perceptron Learning – Solved Example

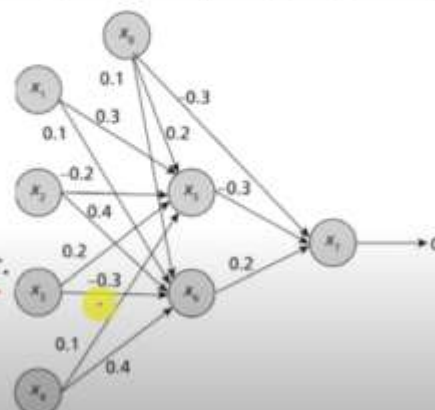
x_1	x_2	x_3	x_4	w_{15}	w_{16}	w_{25}	w_{26}	w_{35}	w_{36}	w_{45}	w_{46}	w_{57}	w_{67}	θ_5	θ_6	θ_7
1	1	0	1	0.3	0.1	-0.2	0.4	0.2	-0.3	0.1	0.4	-0.3	0.2	0.2	0.1	-0.3

3. Calculate Error = $O_{\text{desired}} - O_{\text{Estimated}}$

So, error for this network is:

$$\text{Error} = O_{\text{desired}} - O_7 = 1 - 0.419 = 0.581$$

So, we need to back propagate to reduce the error.



3. Calculate Error = $O_{\text{desired}} - O_{\text{Estimated}}$

So, error for this network is:

$$\text{Error} = O_{\text{desired}} - O_7 = 1 - 0.419 = 0.581$$

So, we need to back propagate to reduce the error.

x_1

0.

x_2

Step 2: Backward Propagation

1. Calculate Error at each node

For each unit k in the output layer, calculate:

$$\checkmark \text{Error}_k = O_k (1 - O_k) (O_{desired} - O_k)$$

For each unit j in the hidden layer, calculate:

$$\checkmark \text{Error}_j = O_j (1 - O_j) \sum_k \text{Error}_k w_{jk}$$

Multi-Layer Perceptron Learning – Solved Example

x_1	x_2	x_3	x_4	w_{15}	w_{16}	w_{25}	w_{26}	w_{35}	w_{36}	w_{45}	w_{46}	w_{57}	w_{67}	θ_5	θ_6	θ_7
1	1	0	1	0.3	0.1	-0.2	0.4	0.2	-0.3	0.1	0.4	-0.3	0.2	0.2	0.1	-0.3

Error Calculation for Each Unit in the Output Layer and Hidden Layer

For Output Layer Unit _k	Error _k
x_7	$\text{Error}_7 = O_7 (1 - O_7) (Y_n - O_7)$ $= 0.419 \times (1 - 0.419) \times (1 - 0.419) = 0.141 \checkmark$
For Hidden Layer Unit _j	Error _j
x_6	$\text{Error}_6 = O_6 (1 - O_6) \sum_k \text{Error}_k w_{jk} = O_6 (1 - O_6) \text{Error}_7 w_{67}$ $= 0.769 (1 - 0.769) \times 0.2 \times 0.141 = 0.005$
x_5	$\text{Error}_5 = O_5 (1 - O_5) \sum_k \text{Error}_k w_{jk} = O_5 (1 - O_5) \text{Error}_7 w_{57}$ $= 0.599 (1 - 0.599) \times 0.141 \times -0.3 = -0.0101$

For Output Layer Unit _k	Error _k
x_7	$\text{Error}_7 = O_7 (1 - O_7) (Y_n - O_7)$ $= 0.419 \times (1 - 0.419) \times (1 - 0.419) = 0.141$
For Hidden Layer Unit _j	Error _j
x_6	$\text{Error}_6 = O_6 (1 - O_6) \sum_k \text{Error}_k w_{jk} = O_6 (1 - O_6) \text{Error}_7 w_{67}$ $= 0.769 (1 - 0.769) \times 0.2 \times 0.141 = 0.005$
x_5	$\text{Error}_5 = O_5 (1 - O_5) \sum_k \text{Error}_k w_{jk} = O_5 (1 - O_5) \text{Error}_7 w_{57}$ $= 0.599 (1 - 0.599) \times 0.141 \times -0.3 = -0.0101$

x_1	x_2	x_3	x_4	w_{15}	w_{16}	w_{25}	w_{26}	w_{35}	w_{36}
1	1	0	1	0.3	0.1	-0.2	0.4	0.2	-0.3

2. Update weight using the below formula:

Learning rate $\alpha = 0.8$. ✓

$$w_{ij} = w_{ij} + \alpha \times \text{Error}_j \times O_i$$

The updated weights and bias

2. Update weight using the below formula:

Learning rate $\alpha = 0.8$. ✓

$$w_{ij} = w_{ij} + \alpha \times \text{Error}_j \times O_i$$

The updated weights and bias

w_{ij}	$w_{ij} = w_{ij} + \alpha \times \text{Error}_j \times O_i$	New Weight
w_{15}	$w_{15} = w_{15} + 0.8 \times \text{Error}_5 \times O_1$ $= 0.3 + 0.8 \times -0.0101 \times 1$	0.292
w_{16}	$w_{16} = w_{16} + 0.8 \times \text{Error}_6 \times O_1$ $= 0.1 + 0.8 \times 0.005 \times 1$	0.104
w_{25}	$w_{25} = w_{25} + 0.8 \times \text{Error}_5 \times O_2$ $= -0.2 + 0.8 \times -0.0101 \times 1$	-0.208
w_{26}	$w_{26} = w_{26} + 0.8 \times \text{Error}_6 \times O_2$ $= 0.4 + 0.8 \times 0.005 \times 1$	0.404

1:

w_{ij}	$w_{ij} = w_{ij} + \alpha \times \text{Error}_j \times O_i$	New Weight
w_{35}	$w_{35} = w_{35} + 0.8 \times \text{Error}_5 \times O_3$ $= 0.2 + 0.8 \times -0.0101 \times 0$	0.2
w_{36}	$w_{36} = w_{36} + 0.8 \times \text{Error}_6 \times O_3$ $= -0.3 + 0.8 \times 0.005 \times 0$	-0.3
w_{45}	$w_{45} = w_{45} + 0.8 \times \text{Error}_5 \times O_4$ $= 0.1 + 0.8 \times -0.0101 \times 1$	0.092
w_{46}	$w_{46} = w_{46} + 0.8 \times \text{Error}_6 \times O_4$ $= 0.4 + 0.8 \times 0.005 \times 1$	0.404
w_{57}	$w_{57} = w_{57} + 0.8 \times \text{Error}_7 \times O_5$ $= -0.3 + 0.8 \times 0.141 \times 0.599$	-0.232
w_{67}	$w_{67} = w_{67} + 0.8 \times \text{Error}_7 \times O_6$ $= 0.2 + 0.8 \times 0.141 \times 0.769$	0.287

Update bias using the below formula:

$$\theta_j = \theta_j + \alpha \times \text{Error}_j$$

θ_j	$\theta_j = \theta_j + \alpha \times \text{Error}_j$	New Bias
θ_5	$\theta_5 = \theta_5 + \alpha \times \text{Error}_5$ $= 0.2 + 0.8 \times -0.0101$	0.192
θ_6	$\theta_6 = \theta_6 + \alpha \times \text{Error}_6$ $= 0.1 + 0.8 \times 0.005$	0.104
θ_7	$\theta_7 = \theta_7 + \alpha \times \text{Error}_7$ $= -0.3 + 0.8 \times 0.141$	-0.187

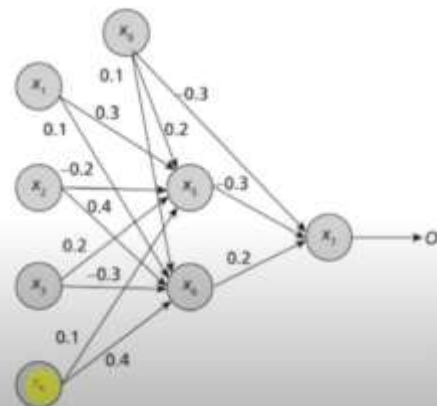
Iteration 2

Now, with the updated weights and biases:

1. Calculate Input and Output in the Input Layer

Net Input and Output Calculation

Input Layer	I_j	O_j
x_1	1	1
x_2	1	1
x_3	0	0
x_4	1	1



2. Calculate Net Input and Output in the Hidden Layer and Output Layer

Net Input and Output Calculation in the Hidden Layer and Output Layer

Unit j	Net Input I_j	Output O_j
x_5	$I_5 = x_1 \times w_{15} + x_2 \times w_{25} + x_3 \times w_{35} + x_4 \times w_{45} + x_0 \times \theta_5$ $I_5 = 1 \times 0.292 + 1 \times -0.208 + 0 \times 0.2 + 1 \times 0.092 + 1 \times 0.192 = 0.368$	$O_5 = \frac{1}{1 + e^{-I_5}} = \frac{1}{1 + e^{-0.368}} = 0.591$
x_6	$I_6 = x_1 \times w_{16} + x_2 \times w_{26} + x_3 \times w_{36} + x_4 \times w_{46} + x_0 \times \theta_6$ $I_6 = 1 \times 0.292 + 1 \times 0.404 + 0 \times -0.3 + 1 \times 0.404 + 1 \times 0.104 = 1.204$	$O_6 = \frac{1}{1 + e^{-I_6}} = \frac{1}{1 + e^{-1.204}} = 0.7692$
x_7	$I_7 = O_5 \times w_{57} + O_6 \times w_{67} + x_0 \times \theta_7$ $I_7 = 0.591 \times -0.232 + 0.7692 \times 0.287 + 1 \times -0.187 = -0.326$	$O_7 = \frac{1}{1 + e^{-I_7}} = \frac{1}{1 + e^{-0.326}} = 0.474$

- The output we receive in the network at node 7 is 0.474

$$\text{Error} = 1 - 0.474 = 0.526$$

- Now, when we compare the error, we get in the previous iteration and in the current iteration,

$$\text{Error reduced is } 0.581 - 0.526 = 0.055$$

- It is visible that the network has learnt and reduced the error by 0.055.
- Thus, the training is continued for a predefined number of epochs or until the training error is reduced below a threshold value.

2.Radial Basis Function

3. Apply the concept of Radial Basis Functions in machine learning and how are they used in practice.

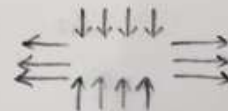
Radial Basis Functions:

- It is the mathematical functions whose value depends solely on the distance between an input data & fixed center point
- It has only one hidden nodes.
- It is applicable when the data is not linearly separable.



- It consists of 9 steps:

- Increase the dimensionality (2D-3D), but it is optional based on requirement
- Expand the direction (horizontal)
compress the direction (vertical)



How RBF does this?

- * Consider one center randomly.
- * Draw concentric circles.

Resultant:

ooooo
ooooo

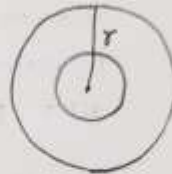
- * To expand/compress, we use 3 functions:

(i) Multiquadric:

$$\phi(r) = (r^2 + c^2)^{1/2}, \quad c > 0 \Rightarrow \text{constant.}$$

(ii) Inverse Multiquadric:

$$\phi(r) = \frac{1}{(r^2 + c^2)^{1/2}}$$



(iii) Gaussian function:

$$\phi(r) = \exp\left[-\frac{r^2}{2\sigma^2}\right]$$

- * The Gaussian function is the most commonly used type.

- * Applications of Radial Basis Functions:

- (i) Radial Basis Function Networks $\rightarrow$ used for function approximation, regression, and classification tasks.
- (ii) Support vector machines (SVMs) with RBF kernel $\rightarrow$ RBF kernel maps data into higher dimensions to make it linearly separable.
- (iii) Image processing $\rightarrow$ used for image interpolation & object recognition.

3. Radial Basis Function Network

<https://www.geeksforgeeks.org/what-are-radial-basis-function-neural-networks/>

Curse of Dimensionality in Machine Learning

Last Updated : 11 Dec, 2024

Curse of Dimensionality in Machine Learning arises when working with high-dimensional data, leading to increased computational complexity, overfitting, and spurious correlations.

Techniques like dimensionality reduction, feature selection, and careful model design are essential for mitigating its effects and improving algorithm performance. Navigating this challenge is crucial for unlocking the potential of high-dimensional datasets and ensuring robust machine-learning solutions.

What is Curse of Dimensionality?

- **Curse of Dimensionality** refers to the phenomenon where the efficiency and effectiveness of algorithms deteriorate as the dimensionality of the data increases exponentially.
- In high-dimensional spaces, data points become sparse, making it challenging to discern meaningful patterns or relationships due to the vast amount of data required to adequately sample the space.
- Curse of Dimensionality significantly impacts [machine learning algorithms](#) in various ways. It leads to increased computational complexity, longer training times, and higher resource requirements. Moreover, it escalates the risk of overfitting and spurious correlations, hindering the algorithms' ability to generalize well to unseen data.

How to Overcome the Curse of Dimensionality?

To overcome the curse of dimensionality, you can consider the following strategies:

1. Dimensionality Reduction Techniques:

- **Feature Selection:** Identify and select the most relevant features from the original dataset while discarding irrelevant or redundant ones. This reduces the dimensionality of the data, simplifying the model and improving its efficiency.
- **Feature Extraction:** Transform the original high-dimensional data into a lower-dimensional space by creating new features that capture the essential information. Techniques such as [Principal Component](#)

Analysis (PCA) and t-distributed Stochastic Neighbor Embedding (t-SNE) are commonly used for feature extraction.

2. Data Preprocessing:

- **Normalization:** Scale the features to a similar range to prevent certain features from dominating others, especially in distance-based algorithms.
- **Handling Missing Values:** Address missing data appropriately through imputation or deletion to ensure robustness in the model training process.

5. Interpolation and basis function

<https://www.geeksforgeeks.org/interpolation-in-machine-learning/>

6. Support Vector Machine

Support Vector Machine – Introduction

- Support Vector Machine or SVM is one of the most popular Supervised Learning algorithms, which is used for Classification as well as Regression problems.
- However, primarily, it is used for Classification problems in Machine Learning.
- The goal of the SVM algorithm is to create the best line or decision boundary that can segregate n-dimensional space into classes so that we can easily put the new data point in the correct category in the future.
- This best decision boundary is called a hyperplane.

Support Vector Machine – Hyperplane

- There can be multiple lines/decision boundaries to segregate the classes in n-dimensional space, but we need to find out the best decision boundary that helps to classify the data points.
- This best boundary is known as the hyperplane of SVM.



Support Vector Machine – Support Vectors:

- The data points or vectors that are the closest to the hyperplane and which affect the position of the hyperplane are termed as Support Vector.
- Since these vectors support the hyperplane, hence called a Support vector.

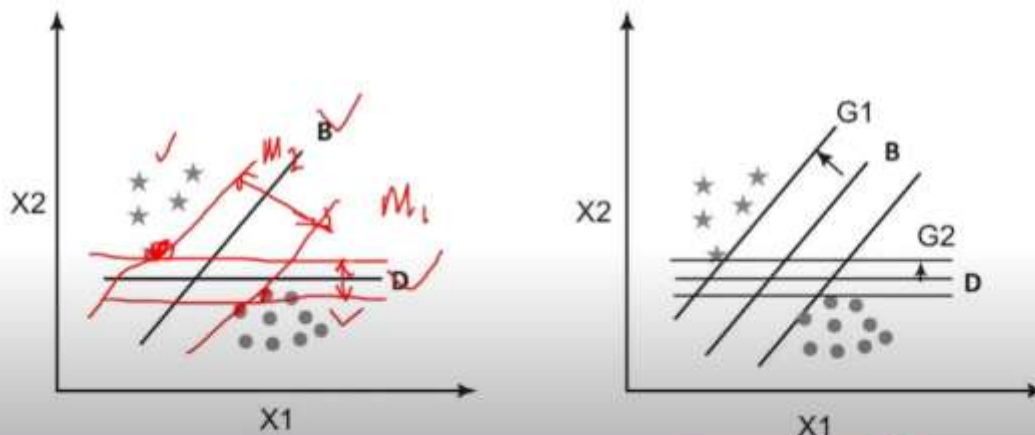
Support Vector Machine – Types of SVM

SVM can be of two types:

1. **Linear SVM:** Linear SVM is used for linearly separable data, which means if a dataset can be classified into two classes by using a single straight line, then such data is termed as linearly separable data, and classifier is used called as Linear SVM classifier.
2. **Non-linear SVM:** Non-Linear SVM is used for non-linearly separated data, which means if a dataset cannot be classified by using a straight line, then such data is termed as non-linear data and classifier used is called as Non-linear SVM classifier.

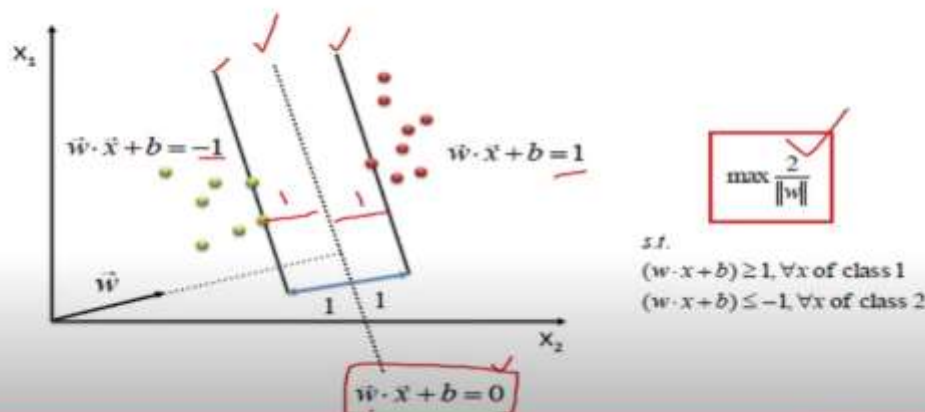
Support Vector Machine – Types of SVM

- Linear SVM:



Support Vector Machine – Types of SVM.

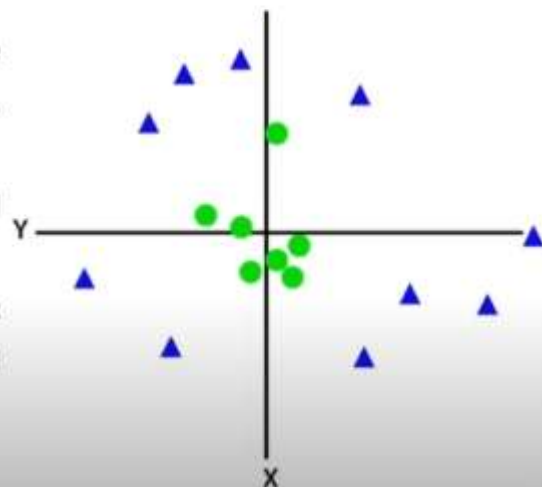
How to find Mean Me



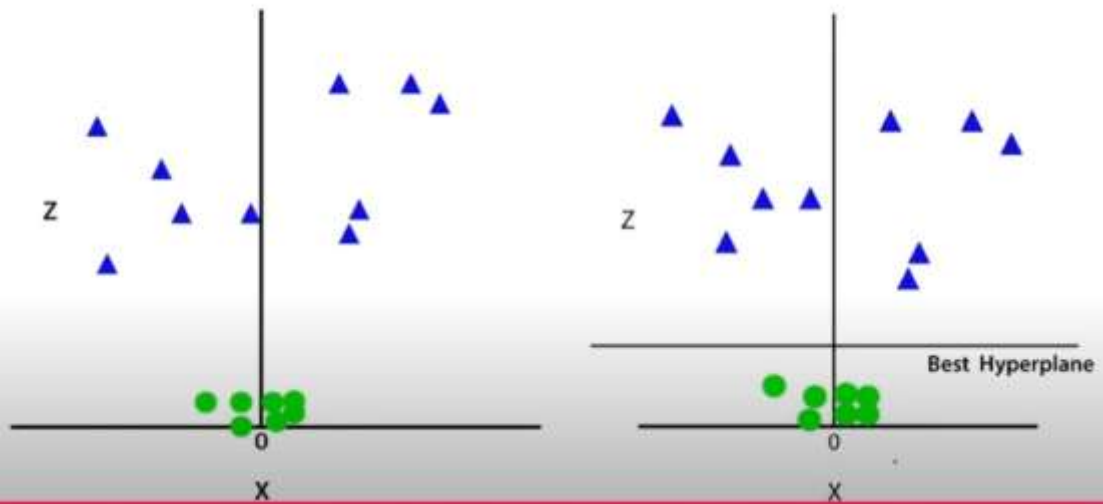
Support Vector Machine – Types of SVM

- Non-Linear SVM:

- If data is linearly arranged, then we can separate it by using a straight line, but for non-linear data, we cannot draw a single straight line.
- So to separate these data points, we need to add one more dimension.
- For linear data, we have used two dimensions x and y , so for non-linear data, we will add a third dimension z .
- It can be calculated as: $z = x^2 + y^2$



Support Vector Machine – Types of SVM



UNIT-3

1. Learning with Tree

1. Learning with Trees?

Learning with trees involves constructing a decision tree model from training data. The goal is to create a model that can make predictions by learning simple decision rules inferred from the data features.

2. Key Concepts in Learning with Trees

a) Supervised Learning

Decision trees are used in **supervised learning**, where the model is trained on labeled data (input features and corresponding output labels).

For **classification**, the output is a class label (e.g., "Yes" or "No").

For **regression**, the output is a continuous value (e.g., house price).

b) Tree Structure

The tree consists of **nodes** (decisions based on features), **branches** (outcomes of decisions), and **leaf nodes** (final predictions).

The tree is built by recursively splitting the data into subsets based on feature values.

c) Splitting Criteria

The algorithm selects the best feature to split the data at each node. Common criteria include:

Gini Impurity (for classification): Measures the likelihood of misclassification.

Entropy (for classification): Measures the disorder in the data.

Information Gain: The reduction in entropy after a split.

Variance Reduction (for regression): Minimizes the variance of the target variable.

d) Stopping Criteria

The tree stops growing when one of the following conditions is met:

Maximum depth is reached.

Minimum number of samples in a leaf node.

No further improvement in splitting criteria.

e) Pruning

To avoid overfitting, the tree can be pruned by removing unnecessary branches after construction.

3. Advantages of Learning with Trees

Interpretability: Trees are easy to visualize and understand.

Handles Non-Linearity: Can model complex, non-linear relationships.

Requires Little Preprocessing: No need for feature scaling or normalization.

Handles Mixed Data Types: Works with both numerical and categorical data.

4. Disadvantages of Learning with Trees

Overfitting: Trees can become too complex and fit the training data too closely.

Instability: Small changes in data can lead to completely different trees.

Bias: Trees can be biased if some classes dominate.

Poor Generalization: May not perform well on unseen data without proper tuning.

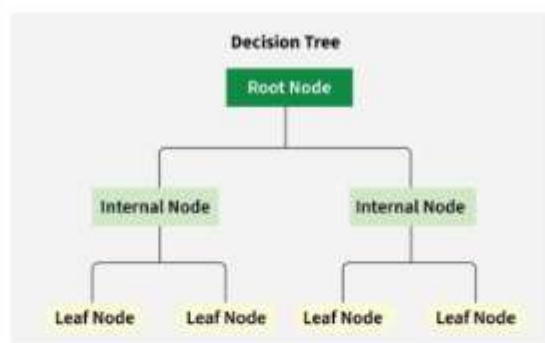
2. Decision Tree

Decision Trees

A **decision tree** is a supervised machine learning algorithm used for both classification and regression tasks. It is a tree-like model of decisions and their possible consequences, where each internal node represents a decision based on a feature, each branch represents the outcome of that decision, and each leaf node represents a final output (class label or continuous value).

Key Components of a Decision Tree:

1. **Root Node:** The topmost node that represents the entire dataset.
2. **Internal Nodes:** Nodes that split the data based on a feature.
3. **Branches:** Outcomes of a decision (e.g., "Yes" or "No").
4. **Leaf Nodes:** Terminal nodes that represent the final decision or prediction.
5. **Splitting:** The process of dividing a node into sub-nodes based on a feature.
6. **Pruning:** Removing unnecessary branches to avoid overfitting.



How a Decision Tree Works:

1. **Start at the Root Node:** The algorithm selects the best feature to split the data.
2. **Split the Data:** The dataset is divided into subsets based on the feature's value.
3. **Repeat:** The process is repeated recursively for each subset until a stopping criterion is met (e.g., maximum depth, minimum samples per leaf).
4. **Assign a Label:** Once a leaf node is reached, the majority class (for classification) or average value (for regression) is assigned.

Advantages of Decision Trees:

1. Easy to understand and interpret (visualizable).
2. Can handle both numerical and categorical data.
3. Requires little data preprocessing (e.g., no need for scaling).
4. Can model non-linear relationships.

Disadvantages of Decision Trees:

1. Prone to overfitting, especially with deep trees.
2. Sensitive to small changes in the data.
3. Can create biased trees if some classes dominate.
4. May not perform well with imbalanced datasets.

Applications:

- Customer segmentation
- Fraud detection
- Medical diagnosis
- Predictive modeling
- Recommendation systems

3. Constructing Decision Tree

📖 What Is a Decision Tree?

A **Decision Tree** is a tree-like model used for classification and regression. It recursively splits the data into subsets based on feature values to make predictions.

✓ Steps to Construct a Decision Tree

1. Select the Best Attribute to Split

At each node, choose the attribute that best splits the dataset.

Common splitting criteria:

- **Gini Impurity** (used in CART)
- **Information Gain** (used in ID3)
- **Gain Ratio** (used in C4.5)
- **Mean Squared Error** (for regression)

The goal is to maximize information gain or minimize impurity.

2. Split the Dataset

Divide the dataset into subsets based on the selected attribute.

- For **categorical features**, create a branch for each category.
 - For **numerical features**, find the optimal threshold to split on (e.g., `feature <= value`).
-

3. Repeat Recursively

For each subset:

- If it meets a **stopping condition** (e.g., pure class, max depth, or min samples), **stop** and assign a leaf node.
 - Else, **repeat** the process for the subset.
-

4. Stopping Criteria

Stop splitting when:

- All records belong to the same class.
 - No features remain.
 - Maximum depth is reached.
 - Number of samples is below a threshold.
-

5. Assign Leaf Nodes

Once a node meets the stopping criteria, it becomes a **leaf node**:

- For **classification**, assign the most common class label.
- For **regression**, assign the average of the target values.

```
from sklearn.tree import
DecisionTreeClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import
train_test_split

# Load dataset
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test =
train_test_split(X, y, random_state=0)

# Create decision tree classifier
clf = DecisionTreeClassifier(criterion='gini',
max_depth=3)
clf.fit(X_train, y_train)
```

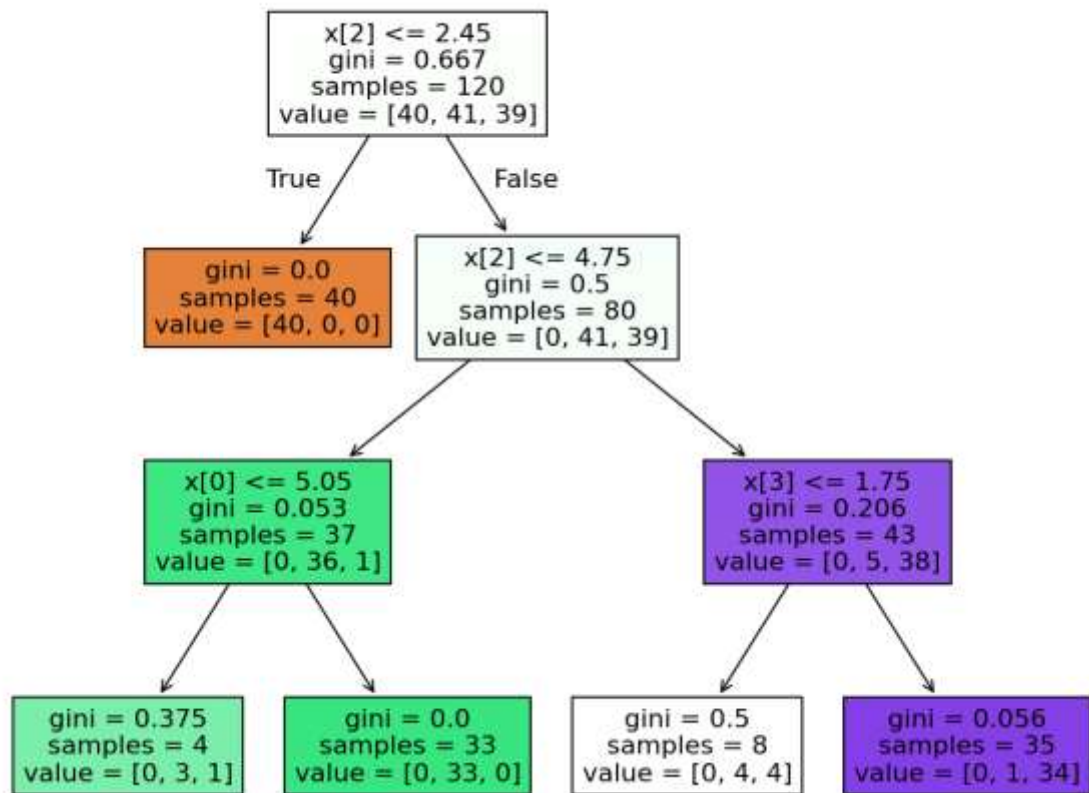
```
# Predict
y_pred = clf.predict(X_test)

from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))
plot_tree(clf, filled=True,
feature_names=load_iris().feature_names)
plt.show()

from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))
plot_tree(clf, filled=True,
feature_names=load_iris().feature_names)
plt.show()
```



4. Classification and regression tree

Classification and Regression Tree (CART) is a type of decision tree algorithm used in machine learning for both **classification** and **regression** tasks. It was introduced by Breiman et al. in 1986 and is one of the most widely used tree-based methods.

◆ 1. What is CART?

CART is a binary decision tree algorithm:

- **Classification Tree:** Used when the target variable is categorical.
- **Regression Tree:** Used when the target variable is continuous.

At each node, CART selects the best split from all possible variables and thresholds, aiming to minimize an impurity measure.

◆ 2. Key Concepts

a. Node: A point in the tree where a decision is made.

b. Root Node: The top node, containing the entire dataset.

c. Leaf/Terminal Node: The final output node — either a class label (classification) or value (regression).

d. Splitting: Dividing a node into two child nodes.

e. Impurity Measures:

- **Classification:** Gini Index, Entropy (Information Gain)
 - **Regression:** Mean Squared Error (MSE), Mean Absolute Error (MAE)
-

◆ 3. How CART Works

For Classification:

- Uses **Gini Index** (default) or **Entropy** to decide splits.
- Chooses the feature and split that result in the largest decrease in impurity.

For Regression:

- Uses **MSE** to determine the best split.
 - Chooses the split that minimizes the squared differences from the mean.
-

◆ 4. Gini Index (for classification)

$$Gini(t) = 1 - \sum_{i=1}^C p_i^2$$

Where p_i is the proportion of class i at node t , and C is the number of classes.

◆ 5. Mean Squared Error (for regression)

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2$$

Where y_i is the actual value and $\bar{y}$ is the mean of the target variable in the node.

◆ 6. Pruning

To avoid overfitting, CART uses **pruning**:

- **Pre-pruning**: Stops splitting early (e.g., max depth, min samples).
- **Post-pruning**: Grows the full tree and then prunes back branches that don't improve performance.

◆ 7. Advantages and Disadvantages

Advantages	Disadvantages
Easy to interpret	Prone to overfitting
Handles both numerical and categorical data	Can be unstable (small changes in data can change the tree)
Non-parametric (no assumptions about data)	Less accurate than ensemble methods like Random Forests

◆ 8. Example Use Cases

- Customer churn prediction (classification)

- House price prediction (regression)
- Medical diagnosis (classification)
- Sales forecasting (regression)

Example

Student	Hours_Studied	Result
A	1	Fail
B	2	Fail
C	3	Pass
D	4	Pass
E	5	Pass

♦ Step-by-Step (Classification)

We will use the **Gini Index** to find the best split.

Step 1: All data (root node)

- Total = 5
- Pass = 3, Fail = 2

$$Gini = 1 - (3/5)^2 - (2/5)^2 = 1 - 0.36 - 0.16 = 0.48$$

Step 2: Try all possible splits

We try splitting based on **Hours_Studied**. The unique values are: 1, 2, 3, 4, 5. We try midpoints between them:

1.5, 2.5, 3.5, 4.5

Try Split: **Hours_Studied <= 1.5**

- Left: [1 → Fail]
 - Gini = 0 (pure node)
- Right: [2,3,4,5 → Fail, Pass, Pass, Pass]
 - Pass = 3, Fail = 1
 - $Gini = 1 - (3/4)^2 - (1/4)^2 = 1 - 0.5625 - 0.0625 = 0.375$

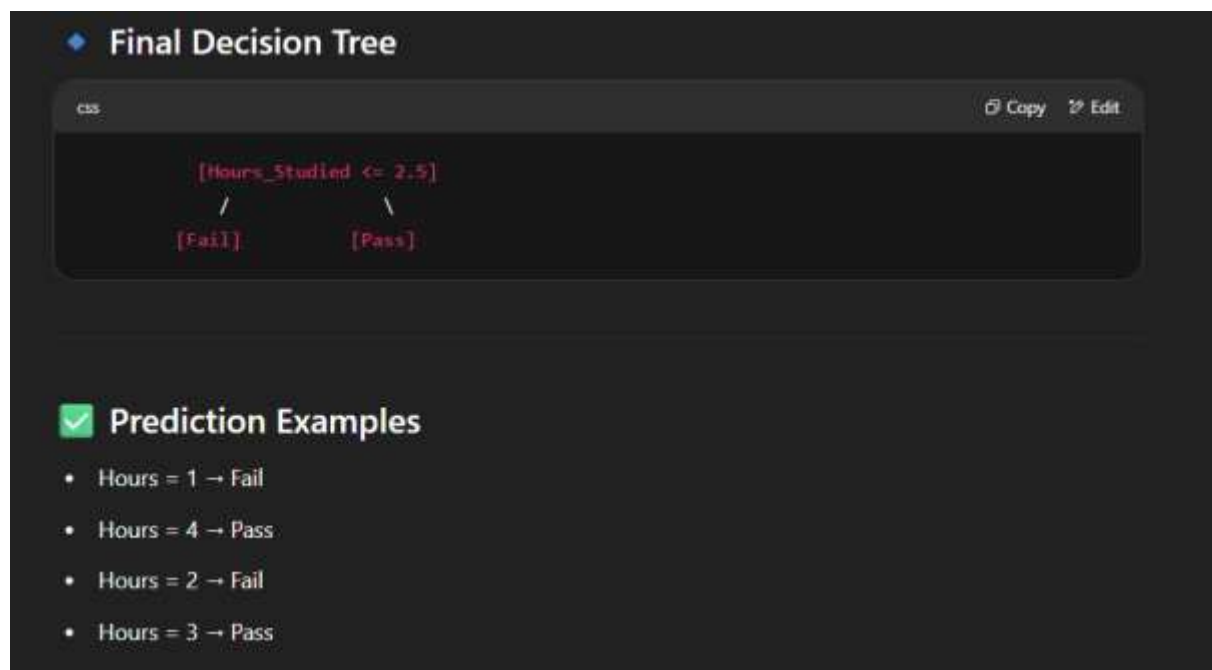
$$\text{Weighted Gini} = (1/5) \times 0 + (4/5) \times 0.375 = 0.3$$

Try Split: **Hours_Studied <= 2.5**

- Left: [1,2 → Fail, Fail] → Gini = 0
- Right: [3,4,5 → Pass] → Gini = 0

Weighted Gini = 0 (Perfect split!)

✅ This is the best split!



5.Ensemble Learning

Ensemble Learning

Ensemble learning combines the predictions of multiple models (called "weak learners" or "base models") to make a stronger, more reliable prediction. The goal is to reduce errors and improve performance.

Types of Ensemble Learning in Machine Learning

There are two main types of ensemble methods:

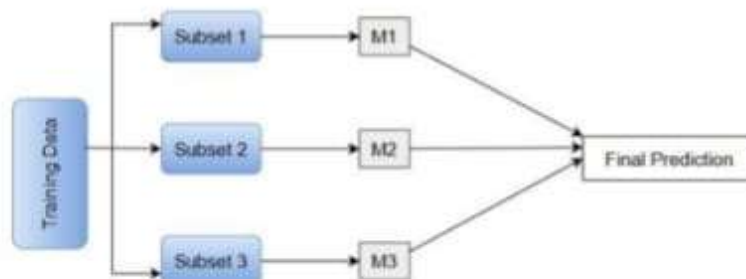
1. Bagging (Bootstrap Aggregating): Models are trained independently on different subsets of the data, and their results are averaged or voted on.
2. Boosting: Models are trained sequentially, with each one learning from the mistakes of the previous model.

27. Compare and contrast bagging, boosting, and stacking

Ans:

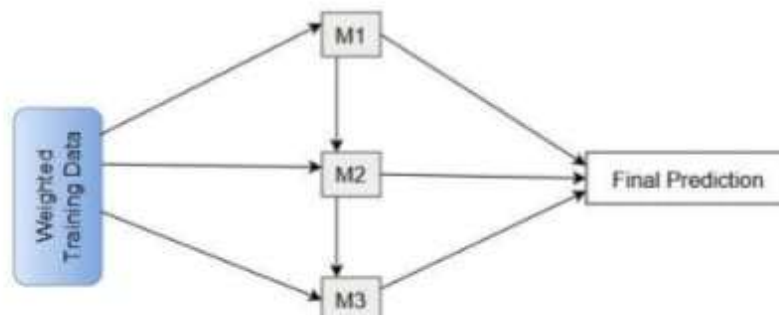
Bagging

Bagging, also known as [bootstrap](#) aggregation, is an ensemble learning technique that combines the benefits of bootstrapping and aggregation to yield a stable model and improve the prediction performance of a machine-learning model.



Boosting

In boosting, we train a sequence of models. Each model is trained on a weighted training set. We assign weights based on the errors of the previous models in the sequence.



1. Bagging Algorithm

Bagging classifier can be used for both regression and classification tasks. Here is an overview of Bagging classifier algorithm:

- **Bootstrap Sampling:** Divides the original training data into 'N' subsets and randomly selects a subset with replacement in some rows from other subsets. This step ensures that the base models are trained on diverse subsets of the data and there is no class imbalance.
- **Base Model Training:** For each bootstrapped sample we train a base model independently on that subset of data. These weak models are trained in parallel to increase computational efficiency and reduce time consumption. We can use different base learners i.e different ML models as base learners to bring variety and robustness.
- **Prediction Aggregation:** To make a prediction on testing data combine the predictions of all base models. For classification tasks it can include majority voting or weighted majority while for regression it involves averaging the predictions.
- **Out-of-Bag (OOB) Evaluation:** Some samples are excluded from the training subset of particular base models during the bootstrapping method. These "out-of-bag" samples can be used to estimate the model's performance without the need for cross-validation.
- **Final Prediction:** After aggregating the predictions from all the base models, Bagging produces a final prediction for each instance.

2. Boosting Algorithm

Boosting is an ensemble technique that combines multiple weak learners to create a strong learner. Weak models are trained in series such that each next model tries to correct errors of the previous model until the entire training dataset is predicted correctly. One of the most well-known boosting algorithms is AdaBoost (Adaptive Boosting). Here is an overview of Boosting algorithm:

- **Initialize Model Weights:** Begin with a single weak learner and assign equal weights to

all training examples.

- Train Weak Learner: Train weak learners on these dataset.
- Sequential Learning: Boosting works by training models sequentially where each model focuses on correcting the errors of its predecessor. Boosting typically uses a single type of weak learner like decision trees.
- Weight Adjustment: Boosting assigns weights to training datapoints. Misclassified examples receive higher weights in the next iteration so that next models pay more attention to them.

6. Different way of combine classifier

Different ways to Combine Classifiers

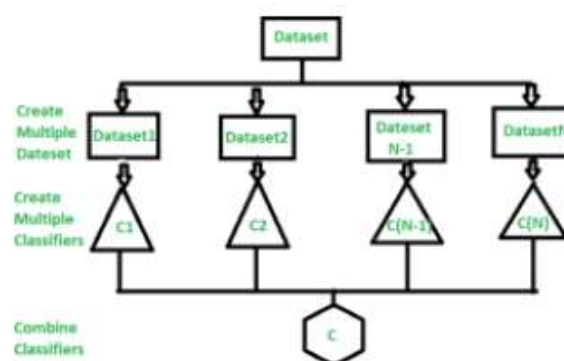
There are multiple ways to combine classifiers, including bagging, boosting, stacking, and classifier ensembles.

Ensemble Classifier | Data Mining

Ensemble learning helps improve machine learning results by combining several models. This approach allows the production of better predictive performance compared to a single model. Basic idea is to learn a set of classifiers (experts) and to allow them to vote.

Advantage : Improvement in predictive accuracy.

Disadvantage : It is difficult to understand an ensemble of classifiers.



Why do ensembles work?

Dietterich(2002) showed that ensembles overcome three problems –

Statistical Problem –

The Statistical Problem arises when the hypothesis space is too large for the amount of available data. Hence, there are many hypotheses with the same accuracy on the data and the learning algorithm chooses only one of them! There is a risk that the accuracy of the chosen hypothesis is low on unseen data!

Computational Problem –

The Computational Problem arises when the learning algorithm cannot guarantee finding the best hypothesis.

Representational Problem –

The Representational Problem arises when the hypothesis space does not contain any good approximation of the target class(es).

Main Challenge for Developing Ensemble Models?

The main challenge is not to obtain highly accurate base models, but rather to obtain base models which make different kinds of errors. For example, if ensembles are used for classification, high accuracies can be accomplished if different base models misclassify different training examples, even if the base classifier accuracy is low.

Methods for Independently Constructing Ensembles –

☐ Majority Vote

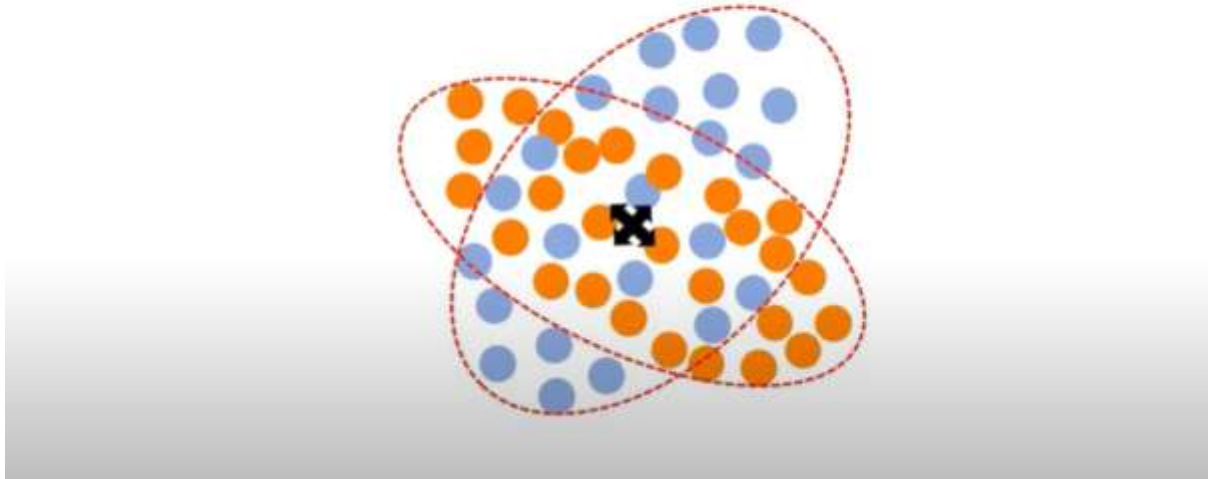
- ❑ Bagging and Random Forest
- ❑ Randomness Injection
- ❑ Feature-Selection Ensembles
- ❑ Error-Correcting Output Coding

Methods for Coordinated Construction of Ensembles –

- ❑ Boosting
- ❑ Stacking

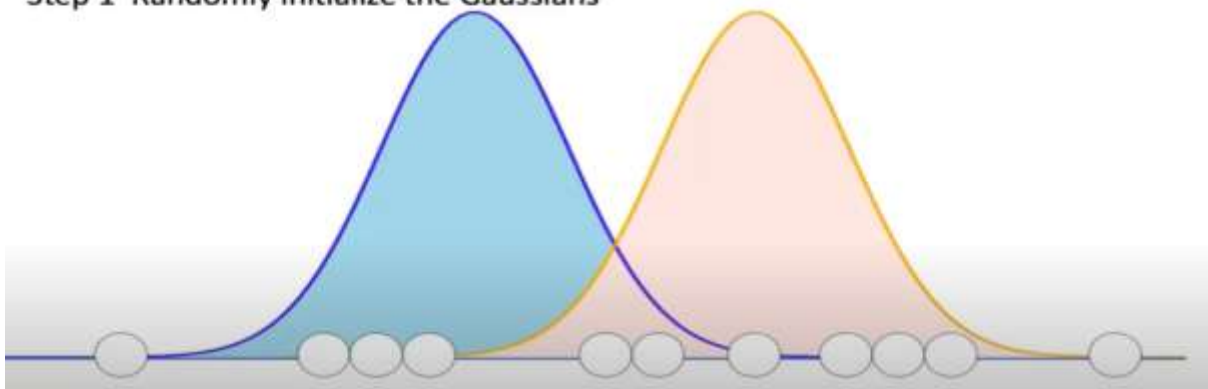
7. Gaussian Mixture Models –

Gaussian Mixture Models



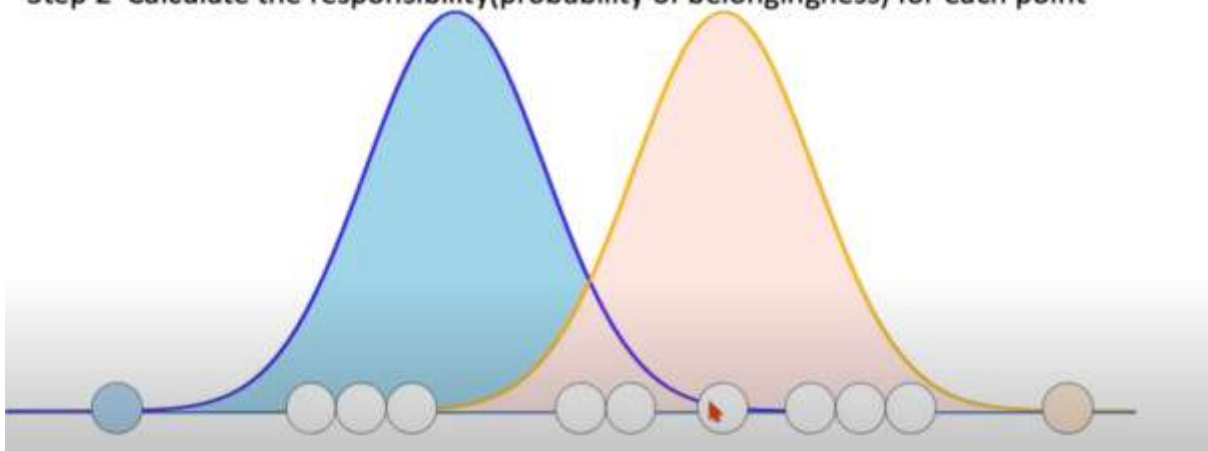
Gaussian Mixture Models

Step 1 Randomly initialize the Gaussians



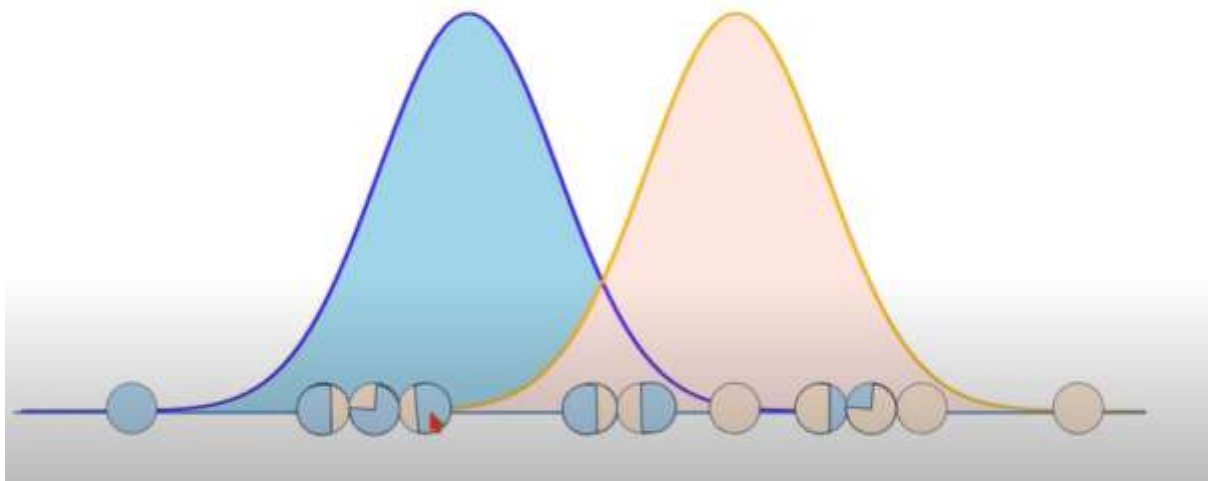
Gaussian Mixture Models

Step 2 Calculate the responsibility(probability of belongingness) for each point



Gaussian Mixture Models

sixsigma
for dummies



Gaussian Mixture Models

sixsigma
for dummies

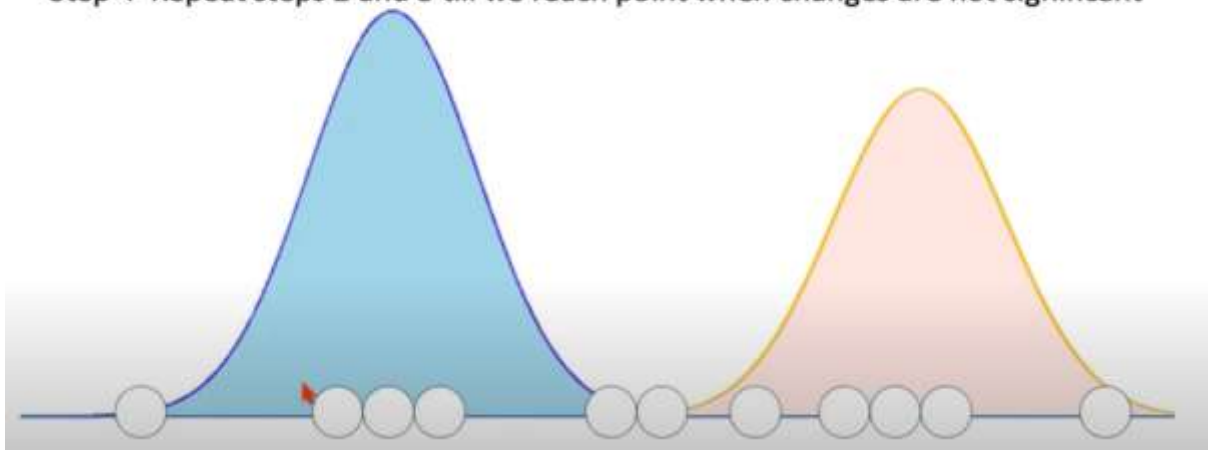
Step 3 Considering the responsibility calculated in the previous step, figure out more appropriate Gaussians



Gaussian Mixture Models

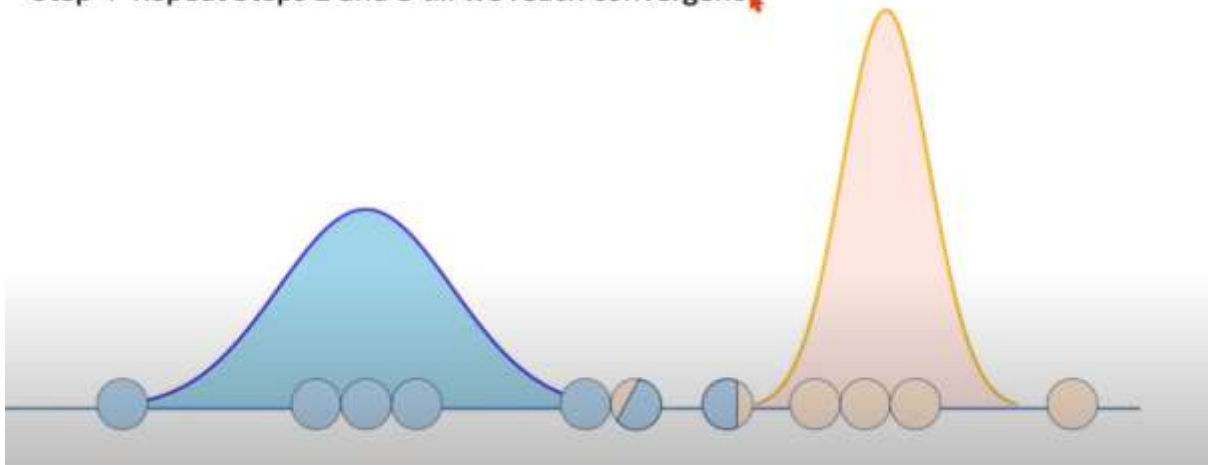
sixsigma
for dummies

Step 4 Repeat steps 2 and 3 till we reach point when changes are not significant



Gaussian Mixture Models

Step 4 Repeat Steps 2 and 3 till we reach convergence



Gaussian Mixture Models

Like K-Means this is an iterative approach

Unlike K-Means, GMM is a Soft Clustering Technique based on probabilistic model which uses EM algorithm



8. Nearest Neighbor Methods

Classification (predicting classes)

- k -nearest neighbors algorithm (k -NN)
- Example: Predicting Movie Genre

IMDb Rating	Duration	Genre
8.0 (Mission Impossible)	160	Action
6.2 (Gadar 2)	170	Action
7.2 (Rocky & Rani)	168	Comedy
8.2 (OMG 2)	155	Comedy

- Now predict the genre of "Barbie" movie with IMDb rating 7.4 and duration 114 minutes.

Step 1: Calculate Distances

Calculate the Euclidean distance between the new movie and each movie in the dataset.

$$\text{Distance to } (8.0, 160) = \sqrt{((7.4 - 8.0)^2 + (114 - 160)^2)} = \sqrt{(0.36 + 2116)} \approx 46.00$$

$$\text{Distance to } (6.2, 160) = \sqrt{((7.4 - 6.2)^2 + (114 - 170)^2)} = \sqrt{(1.44 + 3136)} \approx 56.01$$

$$\text{Distance to } (7.2, 168) = \sqrt{((7.4 - 7.2)^2 + (114 - 168)^2)} = \sqrt{(0.04 + 2916)} \approx 54.00$$

$$\text{Distance to } (8.2, 155) = \sqrt{((7.4 - 8.2)^2 + (114 - 155)^2)} = \sqrt{(0.64 + 1681)} \approx 41.00$$

$K=1$ $K=3$

Step 2: Select K Nearest Neighbors

A, C, C

Step 3: Majority Voting (Classification)

9. KMeans Clustering Algorithm

K-Means Algorithm for Clustering

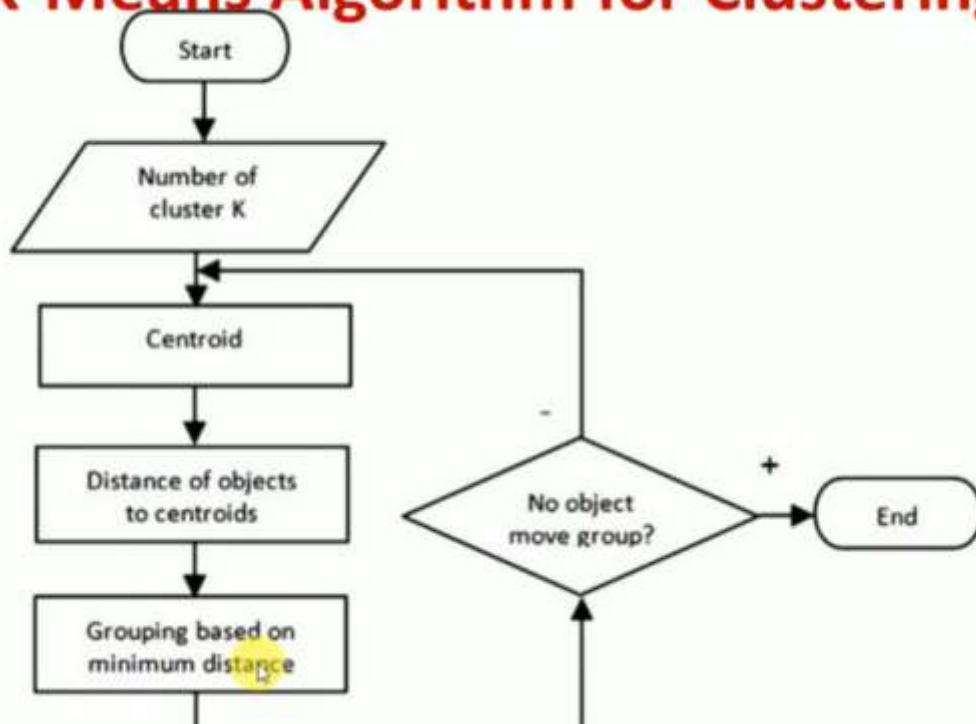
- kMeans algorithm is an unsupervised learning algorithm
- Given a data set of items, with certain features, and values for these features, the algorithm will categorize the items into k groups or clusters of similarity.
- To calculate the similarity, we can use the Euclidean distance, Manhattan distance, Hamming distance, Cosine distance as measurement.

Here is the pseudocode for implementing a K-means algorithm.

Input: Algorithm K-Means (K number of clusters, D list of data points)

1. Choose K number of random data points as initial centroids (cluster centers).
2. Repeat till cluster centers stabilize:
 - a. Allocate each point in D to the nearest of Kth centroids.
 - b. Compute centroid for the cluster using all points in the cluster.

K-Means Algorithm for Clustering



Advantages of K-Means Algorithm

1. K-means algorithm is simple, easy to understand, and easy to implement.
2. It is also efficient, in which the time taken to cluster K-means rises linearly with the number of data points.
3. No other clustering algorithm performs better than K-means.

Disadvantages of K-Means Algorithm

1. The user needs to specify an initial value of K.
2. The process of finding the clusters may not converge.
3. It is not suitable for discovering clusters that are not hyper ellipsoids or hyper spheres).

Eg: <https://www.youtube.com/watch?v=KzJORp8bgqs>

Unit-4

1.Dimensionality Reduction

Dimensionality Reduction:

Dimensionality reduction is the process of reducing the number of features (or dimensions) in a dataset while retaining as much information as possible.

There are several techniques for dimensionality reduction, including principal component analysis (PCA), singular value decomposition (SVD), and linear discriminant analysis (LDA). Each technique uses a different method to project the data onto a lower-dimensional space while preserving important information.

Let's take an example to explain this better:

Imagine you are building a machine learning model to predict **house prices** based on features like the number of **bedrooms**, **square footage**, **location**, **age of the house**, **number of bathrooms**, and so on. If you have too many features like additional ones for each room's condition, flooring type, or neighborhood amenities, your dataset can become very large and complex.

Before Dimensionality Reduction

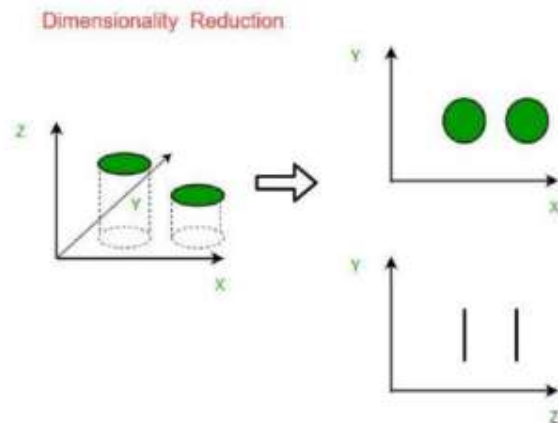
With too many features, your model may become slow to train, and it might also pick up unnecessary details or noise. For example, if the flooring type doesn't significantly impact house prices, it might lead the model to make less accurate predictions, especially when the data is noisy or when there are many irrelevant features.

How Dimensionality Reduction Works?

Lets understand how dimensionality Reduction is used with the help of the figure below:

How Dimensionality Reduction Works?

Lets understand how dimensionality Reduction is used with the help of the figure below:



On the left, data points exist in a **3D space** (X, Y, Z), but the Z-dimension appears unnecessary since the data primarily varies along the X and Y axes. The goal of dimensionality reduction is to remove less important dimensions without losing valuable information.

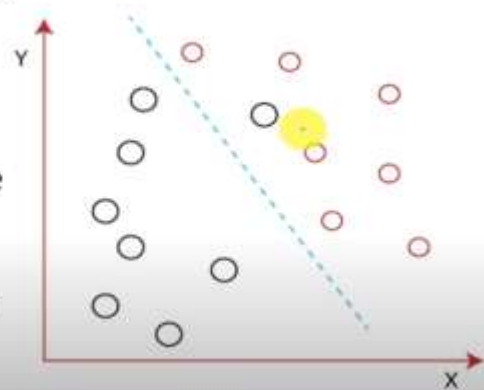
On the right, after reducing the dimensionality, the data is represented in **lower-dimensional spaces**. The top plot (X-Y) maintains the meaningful structure, while the bottom plot (Z-Y) shows that the Z-dimension contributed little useful information. This process makes data analysis more efficient, improving computation speed and visualization while minimizing redundancy.

2. – Linear Discriminant Analysis

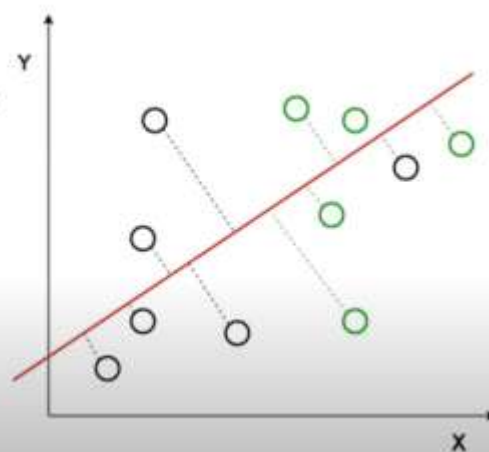
Linear Discriminant Analysis - Introduction

- Linear Discriminant Analysis or Normal Discriminant Analysis or Discriminant Function Analysis is a dimensionality reduction technique that is commonly used for supervised classification problems.
- It is used to project the features in higher dimension space into a lower dimension space.

- Suppose we have two sets of data points belonging to two different classes that we want to classify.
- When the data points are plotted on the 2D plane, there's no straight line that can separate the two classes of the data points completely.



- Here, Linear Discriminant Analysis uses both the axes (X and Y) to create a new axis and projects data onto a new axis in a way to maximize the separation of the two categories and hence, reducing the 2D graph into a 1D graph.



- Two criteria are used by LDA to create a new axis:
 - Maximize the distance between means of the two classes.
 - Minimize the variation within each class.



1. Compute the class means of dependent variable
2. Derive the covariance matrix of the class variable
3. Compute the within class — scatter matrix
($S_1 + S_2$)
4. Compute the between class scatter matrix
5. Compute the Eigen values and eigen vectors
from the within class and between class scatter
matrix

$$\mu_i = \frac{1}{N_i} \sum_{x \in \omega_i} x$$

$$S_i = \sum_{x \in \omega_i} (x - \mu_i)(x - \mu_i)^T$$

$$S_w = S_1 + S_2$$

$$S_B = (\mu_1 - \mu_2)(\mu_1 - \mu_2)^T$$

$$S_w^{-1} S_B w = \lambda w$$

6. Sort the values of eigen values and select
the top k values
7. Find the eigen vectors corresponds to the
top k eigen vectors
8. Obtain the LDA by taking the dot product of
eigen vectors and original data

$$\left(S_w^{-1} S_B - \lambda I \right) \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} = 0$$

3.Principle Component Analysis

Principal Component Analysis (PCA) is a widely used unsupervised linear dimensionality reduction technique. Its primary goal is to transform a dataset with a large number of correlated variables into a smaller set of uncorrelated variables called principal components while retaining as much of the original data's variance as possible. Essentially, PCA identifies the directions (principal components) in the data that capture the most significant variations.

Steps involved in algorithm and Mathematical Derivation

Steps of Principal Component Analysis (PCA)

- Step-1: Calculate Mean of every feature
- Step-2: Calculate the Covariance Matrix
- Step-3: Calculate the Eigenvalues and Eigenvectors
- Step 4: Choose Principal Components
- Step 5: Project Data onto Principal Components

Size (in canal) X_1	No. of Rooms X_2
5	10
3	7
10	15
2	4
$\frac{20}{4}$	$\frac{36}{4}$

Step-1: Calculate Mean:
 $\bar{X}_1 = 5$
 $\bar{X}_2 = 9$

Step-2: Calculate Covariance Matrix:
 $a_{11} = \frac{(5-5)^2 + (3-5)^2 + (10-5)^2 + (2-5)^2}{N-1} = \frac{0 + 4 + 25 + 9}{3} = 12.67$
 $a_{12} = \frac{(5-5)(10-9) + (3-5)(7-9) + (10-5)(15-9) + (2-5)(4-9)}{N-1} = \frac{0 + 12 + 15 + 15}{3} = 16.33$
 $a_{22} = \frac{(10-9)^2 + (7-9)^2 + (15-9)^2 + (4-9)^2}{N-1} = \frac{1 + 4 + 36 + 25}{3} = 22$

Size (in canal) X_1	No. of Rooms X_2
5	10
3	7
10	15
2	4
$\frac{20}{4}$	$\frac{36}{4}$

Step-1: Calculate Mean:
 $\bar{X}_1 = 5$
 $\bar{X}_2 = 9$

Step-2: Calculate Covariance
 $a_{11} = \frac{(5-5)^2 + (3-5)^2 + (10-5)^2 + (2-5)^2}{N-1} = \frac{0 + 4 + 25 + 9}{3} = 12.67$
 $a_{12} = \frac{(5-5)(10-9) + (3-5)(7-9) + (10-5)(15-9) + (2-5)(4-9)}{N-1} = \frac{0 + 12 + 15 + 15}{3} = 16.33$
 $a_{22} = \frac{(10-9)^2 + (7-9)^2 + (15-9)^2 + (4-9)^2}{N-1} = \frac{1 + 4 + 36 + 25}{3} = 22$

Step-3: Calculate Eigenvalues:

$\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$

$\left| \begin{pmatrix} 12.67 - \lambda & 16.33 \\ 16.33 & 22 - \lambda \end{pmatrix} \right| = 0$

$\lambda_1 = 34.3$
 $\lambda_2 = -0.4$

Size (in canal) X_1	No. of Rooms X_2
5	10
3	7
10	15
2	4

Step-4: Calculate Eigenvector:

$\begin{bmatrix} 12.67 - \lambda & 16.33 \\ 16.33 & 22 - \lambda \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$

$\begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = \begin{bmatrix} -1.3256 \\ 1 \end{bmatrix}$

Size (in canal) X_1	No. of Rooms X_2
5	10
3	7
10	15
2	4

Step-5: Project Data onto Principal Components

20×10

Size (in canal) X_1	No. of Rooms X_2	PC
5	10	3.3
3	7	3
10	15	1.8
2	4	1.3

$\begin{bmatrix} 5 & 10 \end{bmatrix} \begin{bmatrix} -1.3256 \\ 1 \end{bmatrix}$

$5 \times -1.3256 + 10 \times 1$
 $3d - 1.3256 + 7 \times 1$

Advantages of PCA

- **Dimensionality Reduction:** Reduces the number of features, simplifying the data and potentially improving the performance of subsequent machine learning algorithms by mitigating the curse of dimensionality.
- **Noise Reduction:** By focusing on the directions of maximum variance, PCA can help filter out noise or less informative dimensions in the data.
- **Feature Extraction:** Creates new, uncorrelated features (principal components) that are linear combinations of the original features. These new features can be more meaningful or easier to interpret in some contexts.
- **Data Visualization:** Reduces high-dimensional data to two or three dimensions, allowing for easier visualization and exploration of data patterns.

Limitations of PCA

- **Linearity Assumption:** PCA assumes that the principal components are linear combinations of the original features. It may not be effective for data with complex, non-linear relationships.
- **Sensitivity to Scaling:** The results of PCA can be sensitive to the scaling of the original features. It's often necessary to standardize or normalize the data before applying PCA.
- **Loss of Information:** Dimensionality reduction inherently involves some loss of information. The goal is to minimize this loss by retaining the most important variance.

Real-World Applications of PCA

PCA is a versatile technique with applications in various fields:

- **Image Compression:** Reducing the dimensionality of image data while retaining essential visual information (e.g., JPEG compression uses a related technique called Discrete Cosine Transform).
- **Face Recognition:** Reducing the dimensionality of face images to extract the most important features for recognition (eigenfaces).
- **Finance:** Reducing the number of correlated financial indicators to identify underlying market trends.
- **Natural Language Processing:** Reducing the dimensionality of word embeddings or document-term matrices.
- **Data Visualization:** Projecting high-dimensional datasets into 2D or 3D for visualization and exploratory data analysis.

.Factor Analysis

4.Independent componenet Analysis

Independent Component Analysis

Independent Component Analysis (ICA) is a technique used to separate mixed signals into their independent, non-Gaussian components. **It aims to find a linear transformation of data that maximizes statistical independence among the components.**

ICA is widely applied in fields like audio, image processing, and biomedical signal analysis to isolate distinct sources from mixed signals

Statistical Independence Concept

Statistical independence refers to the idea that two random variables, X and Y, are independent if knowing one does not affect the probability of the other. Mathematically, this means the joint probability of X and Y is equal to the product of their individual probabilities.

$$P(X \text{ and } Y) = P(X) \cdot P(Y) \text{ or } P(X \cap Y) = P(X) \cdot P(Y)$$

Assumptions in ICA

1. ICA operates under two key assumptions:
2. 1. The source signals are statistically independent of each other.
3. 2. The source signals have non-Gaussian distributions. These assumptions allow ICA to effectively separate mixed signals into independent components, a task that traditional methods like PCA cannot achieve.

Mathematical Representation of ICA

So far, we have discussed about the basics of ICA, but lets jump into the mathematical representation of independent component analysis in brief.

The observed random vector is $X = (x_1, \dots, x_m)^T$, representing the observed data with m components. The hidden components are represented by the random vector $S = (s_1, \dots, s_n)^T$, where n is the number of hidden sources.

Linear Static Transformation

The observed data X is transformed into hidden components S using a linear static transformation representation by the matrix W.

$$S = W^T X$$

Here, W = transformation matrix.

The goal is to transform the observed data x in a way that the resulting hidden components are independent. The independence is measured by some function $F(s_1, \dots, s_n)$.

The task is to find the optimal transformation matrix W that maximizes the independence of the hidden components.

Cocktail Party Problem in ICA

To better understand how Independent Component Analysis (ICA) works, let's look at a classic example known as the **Cocktail Party Problem**

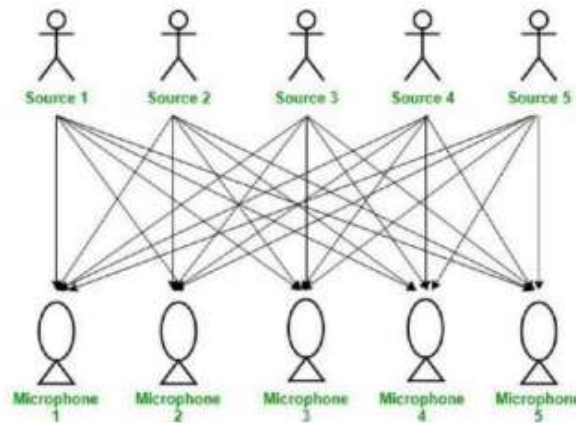
Problem Statement: To extract independent sources' signals from a mixed signal composed of the signals from those sources.

Given: Mixed signal from five different independent sources.

Aim: To decompose the mixed signal into independent sources:

- Source 1
- Source 2
- Source 3
- Source 4
- Source 5

Solution: Independent Component Analysis



Here, there is a party going into a room full of people. There is 'n' number of speakers in that room, and they are speaking simultaneously at the party. In the same room, there are also 'n' microphones placed at different distances from the speakers, which are recording 'n' speakers' voice signals. Hence, the number of speakers is equal to the number of microphones in the room.

Now, using these microphones' recordings, we want to separate all the 'n' speakers' voice signals in the room, given that each microphone recorded the voice signals coming from each speaker of different intensity due to the difference in distances between them.

Decomposing the mixed signal of each microphone's recording into an independent source's speech signal can be done by using the machine learning technique, independent component analysis.

$$[X_1, X_2, \dots, X_n] \Rightarrow [Y_1, Y_2, \dots, Y_n] [X_1, X_2, \dots, X_n] \Rightarrow [Y_1, Y_2, \dots, Y_n]$$

where, $X_1, X_2, \dots, X_n$ are the original signals present in the mixed signal and $Y_1, Y_2, \dots, Y_n$ are the new features and are independent components that are independent of each other.

5. Local Linear Embedding

Locally Linear Embedding

LLE(**Locally Linear Embedding**) is an unsupervised approach designed to transform data from its original high-dimensional space into a lower-dimensional representation, all while striving to retain the essential geometric characteristics of the underlying non-linear feature structure.

How Locally Linear Embedding Captures Relevant Characteristics?

LLE operates in several key steps:

Firstly, it constructs a nearest neighbors graph to capture these local relationships. Then, it optimizes weight values for each data point, aiming to minimize the reconstruction error when expressing a point as a linear combination of its neighbors. This weight matrix reflects the strength of connections between points.

Next, LLE computes a lower dimensional representation of the data by finding eigenvectors of a matrix derived from the weight matrix. These eigenvectors represent the most relevant directions in the reduced space. Users can specify the desired dimensionality for the output space, and LLE selects the top eigenvectors accordingly.

As an illustration, consider a Swiss roll dataset, which is inherently non-linear in its high-dimensional space. LLE, in this case, works to project this complex structure onto a lower-dimensional plane, preserving its distinctive geometric properties throughout the transformation process."

Mathematical Implementation of LLE Algorithm

The key idea of LLE is that locally, in the vicinity of each data point, the data lies approximately on a linear subspace. LLE attempts to unfold or unroll the data while preserving these local linear relationships.

Here is a mathematical overview of the LLE algorithm:

$$\text{Minimize: } \sum_i |x_i - \sum_j w_{ij} x_j|^2$$

$$\text{Subject to : } \sum_j w_{ij} = 1$$

Where:

- x_i represents the i -th data point.
- w_{ij} are the weights that minimize the reconstruction error for data point x_i using its neighbors.

It aims to find a lower-dimensional representation of data while preserving local relationships. The mathematical expression for LLE involves minimizing the reconstruction error of each data point by expressing it as a weighted sum of its k nearest neighbors' contributions. This optimization is subject to constraints ensuring that the weights sum to 1 for each data point. Locally Linear Embedding (LLE) is a dimensionality reduction technique used in machine learning and data analysis. It focuses on preserving local relationships between data points when mapping high-dimensional data to a lower-dimensional space. Here, we will explain the LLE algorithm and its parameters.

Locally Linear Embedding Algorithm

The LLE algorithm can be broken down into several steps:

Neighborhood Selection: For each data point in the high-dimensional space, LLE identifies its k -nearest neighbors. This step is crucial because LLE assumes that each data point can be well approximated by a linear combination of its neighbors.

Weight Matrix Construction: LLE computes a set of weights for each data point to express it as a linear combination of its neighbors. These weights are determined in such a way that the reconstruction error is minimized. Linear regression is often used to find these weights.

Global Structure Preservation: After constructing the weight matrix, LLE aims to find a lower-dimensional representation of the data that best preserves the local linear relationships. It does this by seeking a set of coordinates in the lower-dimensional space for each data point that minimizes a cost function. This [cost function](#) evaluates how well each data point can be represented by its neighbors.

Output Embedding: Once the optimization process is complete, LLE provides the final lower-dimensional representation of the data. This representation captures the essential structure of the data while reducing its dimensionality.

Parameters in LLE Algorithm

LLE has a few parameters that influence its behavior:

k (Number of Neighbors): Explain how to select the optimal number of neighbors, balancing local and global relationships.

Dimensionality of Output Space: Discuss the importance of choosing the right dimensionality for the final representation.

Distance Metric: Introduce different distance metrics like Euclidean or Manhattan and explain their impact.

Regularization: If applicable, briefly mention how regularization helps avoid overfitting.

6.Isomap

Isomap

Isomap | A Non-linear Dimensionality Reduction Technique

Isomap's step-by-step process for non-linear dimensionality reduction.

Isomap (Isometric Mapping) is a powerful technique used in [dimensionality reduction](#) designed to preserve the essential structure of high-dimensional data. Unlike linear techniques like PCA (Principal Component Analysis), Isomap excels at uncovering the non-linear relationships hidden within data, such as those found in images, speech, or biological data. This makes it a valuable tool for machine learning tasks where traditional methods may fall short.

Why Isomap ?

When working with complex, high-dimensional datasets, Isomap offers an advantage over linear methods by capturing non-linear patterns. This ability to retain the geometry of data points in a lower-dimensional space allows for better analysis, visualization, and interpretation of complex datasets.

Understanding Manifold Learning

To fully understand Isomap, it's essential to grasp the concept of manifold learning, which is the basis for this technique

Manifold Learning is a form of [unsupervised learning](#) that aims to reveal the intrinsic structure of high-dimensional data. It treats data as if it lies on a lower-dimensional "manifold" embedded in a higher-dimensional space. Isomap is one such manifold learning technique that preserves the geometric relationships in data as it reduces dimensions

Geodesic Distances and Euclidean Distances

In manifold learning, we distinguish between two types of distances:

- **Geodesic Distance** is the shortest path between two points along the manifold's surface, taking into account its curvature.
- **Euclidean Distance** is the straight-line distance between two points in the original space. Isomap focuses on preserving **geodesic distances**, which reflect the true geometry of the data, unlike Euclidean distances, which may distort the relationships in non-linear datasets

The Concept of Isometric Mapping

At the heart of Isomap is the idea of **Isometric Mapping**, which aims to preserve pairwise distances between points. This ensures that the data's internal structure is retained even as it is reduced to a lower-dimensional representation. Isomap's goal is to ensure that the geodesic distances between points remain as accurate as possible, even when simplifying the data.

How Does Isomap Work?

Now that we understand the core concepts, let's take a step by step look at how Isomap works in practice.

1. **Step 01: Calculate Pairwise Distances:** Begin by computing the Euclidean distances between all pairs of data points.
2. **Step 02: Find Nearest Neighbors:** For each data point, identify its nearest neighbors based on the calculated distances.
3. **Step 03: Create a Neighborhood Graph:** Construct a graph where each point is connected to its nearest neighbors.
4. **Step 04: Calculate Geodesic Distances:** Use algorithms like Floyd-Warshall to determine the shortest paths (geodesic distances) between pairs of points on the graph.
5. **Step 05: Perform Dimensional Reduction:** Apply techniques like Multi-Dimensional Scaling (MDS) to project the data into a lower-dimensional space, preserving the important distances.

Getting started with Isomap Parameters

Isomap comes with several parameters that control its behavior, and understanding these will help you fine tune your application of the technique.

- **n_neighbors:** Specifies the number of nearest neighbors to consider when calculating distances.
- **n_components:** Defines the number of dimensions for the reduced data (usually 2 or 3).
- **eigen_solver:** Determines the algorithm for solving eigenvalues during dimensionality reduction.
- **radius:** Alternatively, you can set a radius to include all points within a certain distance instead of using a fixed number of neighbors.
- **tol:** This tolerance value controls the convergence of the eigenvalue solver.
- **max_iter:** Limits the number of iterations for the eigenvalue solver.
- **path_method:** Determines the algorithm for computing geodesic distances, such as the Floyd-Warshall method.

SVSV Prasad Sanaboina., Asst.Prof

-
- **neighbors_algorithm:** Specifies the method for finding nearest neighbors (e.g., ball tree, kd-tree).
 - **metric:** Defines the distance metric used in nearest neighbor search (e.g., Euclidean, Manhattan)

Choosing the right parameters is essential for optimal results. For example, increasing the number of neighbors can capture more global structure but may require more computation. Experimenting with different values and cross-validating results will help you find the best settings for your dataset.

7. Least Squares Optimization

Least Squares Optimization

Least Squares Optimization is a fundamental technique in machine learning, particularly in regression problems. It aims to minimize the sum of squared differences between observed and predicted values. Here's a breakdown of its role, formulation, and applications:

1. What is Least Squares Optimization?

It is a method for estimating parameters by minimizing the sum of squared residuals (errors) between the predicted outputs of a model and the actual observed values. Mathematically, for a model with parameters θ , the objective is:

$$\min_{\theta} \sum_{i=1}^n (y_i - f(x_i; \theta))^2$$

where:

y_i = true label/observation,
 $f(x_i; \theta)$ = predicted value,
 n = number of data points.

2. Linear Least Squares (Ordinary Least Squares - OLS)

In **linear regression**, the model is $f(x_i; \theta) = \theta^T x_i$, and the optimization problem becomes:

$$\min_{\theta} \|y - X\theta\|_2^2$$

where:

X is the design matrix (each row is a data point),
 y is the vector of observed outputs.

Closed-form Solution:

The optimal parameters θ^* can be derived analytically:

$$\theta^* = (X^T X)^{-1} X^T y$$

Assumptions:

Linear relationship between features and target.
No multicollinearity in X .
Homoscedasticity (constant variance of errors).

3. Nonlinear Least Squares

For nonlinear models (e.g., neural networks, polynomial regression), the objective remains the same, but no closed-form solution exists. Instead, iterative methods are used:

Gradient Descent: Update θ using the gradient of the loss.

Gauss-Newton Method: Approximates the problem as linear at each step.

Levenberg-Marquardt: Combines gradient descent and Gauss-Newton.

4. Weighted Least Squares (WLS)

Used when errors have unequal variance (heteroscedasticity). Weights w_i are assigned to each data point:

$$\min_{\theta} \sum_{i=1}^n w_i (y_i - f(x_i; \theta))^2$$

6. Applications in Machine Learning

1. **Linear/Polynomial Regression:** Predicting continuous outputs.
2. **Support Vector Machines (SVMs):** Least squares SVM variants.
3. **Recommender Systems:** Matrix factorization (e.g., ALS for collaborative filtering).
4. **Neural Networks:** Mean Squared Error (MSE) loss function.
5. **Computer Vision:** Solving geometric problems (e.g., homography estimation).

Evolutionary Learning

Evolutionary learning in machine learning uses algorithms inspired by natural selection and evolution, like genetic algorithms, to find optimal solutions to problems by iteratively improving a population of candidate solutions

Evolutionary learning is a subset of machine learning that takes inspiration from biological evolution to optimize models and solutions. It is particularly useful for complex optimization problems where traditional gradient-based methods may struggle. This learning paradigm is based on principles like **natural selection, mutation, crossover, and survival of the fittest**.

Genetic Algorithm

- A **genetic algorithm** is a **heuristic search** algorithm that is inspired by Charles Darwin's theory of natural evolution.
- This algorithm reflects the process of **natural selection** where the fittest individuals are selected for reproduction in order to produce offspring of the next generation.
- Genetic Algorithms are being widely used in different **real-world applications**, for example, **image processing**, **Designing electronic circuits**, **code-breaking**, and **artificial creativity**.

How Genetic Algorithm works

There are five phases in Genetic Algorithm:

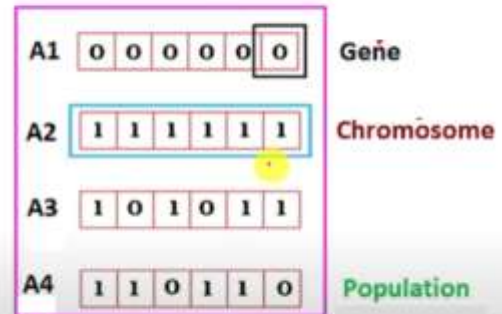
- Initialization
- Fitness Assignment
- Selection
- Crossover (Reproduction)
- Termination



How Genetic Algorithm works

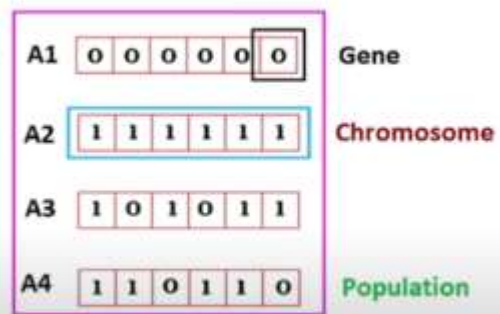
Initial Population

- The process begins with a set of individuals which is called a Population.
- Each individual is a solution to the problem you want to solve known as Chromosome.
- An individual is characterized by a set of parameters (variables) known as Genes.
- Genes are joined into a string to form a Chromosome (solution).



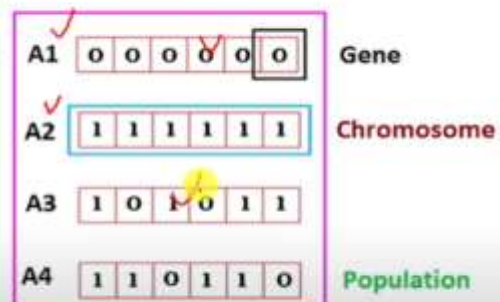
Fitness Function

- The fitness function determines how fit an individual is? (the ability of an individual to compete with other individuals).
- It gives a fitness score to each individual.



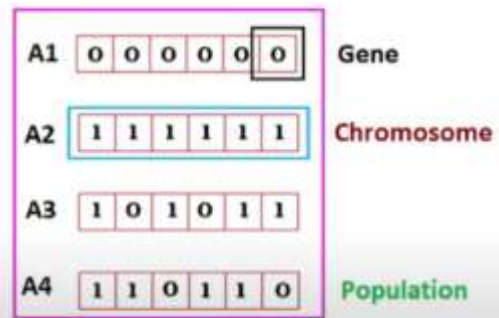
Selection

- The idea of selection phase is to select the fittest individuals and let them pass their genes to the next generation.



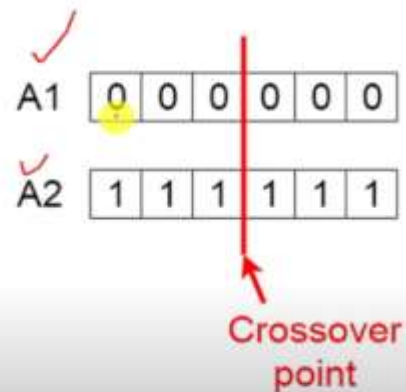
There are three types of Selection methods available, which are:

1. Roulette wheel selection ✓
2. Tournament selection ✓
3. Rank-based selection ✓



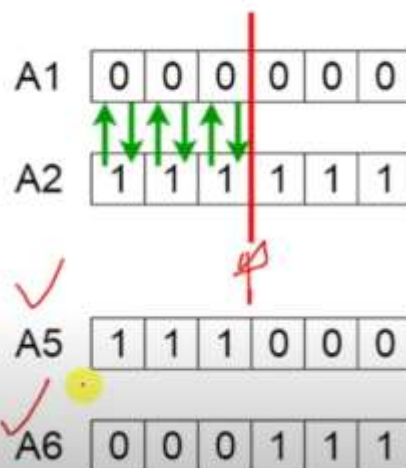
Crossover

- Crossover is the most significant phase in a genetic algorithm.
- For each pair of parents to be mated, a crossover point is chosen at random from within the genes.
- For example, consider the crossover point to be 3 as shown.



Offspring

- Offspring are created by exchanging the genes of parents among themselves until the crossover point is reached.
- The new offspring are added to the population.



Mutation

- In certain new offspring formed, some of their genes can be subjected to a mutation with a low random probability.
- This implies that some of the bits in the bit string can be flipped.

Before Mutation

✓ A5

1	1	1	0	0	0
---	---	---	---	---	---

After Mutation

A5

1	1	0	1	1	0
---	---	---	---	---	---

Types of mutation styles available,

1. Flip bit mutation
2. Gaussian mutation
3. Exchange/Swap mutation

Before Mutation

A5

1	1	1	0	0	0
---	---	---	---	---	---

After Mutation

A5

1	1	0	1	1	0
---	---	---	---	---	---

Termination

- The algorithm terminates if the population has converged (does not produce offspring which are significantly different from the previous generation).

How Genetic Algorithm works

